GIT WORKSHOP

Wolfgang Höfer

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Vorbemerkung

Git ist das aktuell wohl gängigste Versionskontrollsystem auf dem Markt. Allerdings gibt es im Zusammenhang mit *git* so viele verschiedene Aspekte zu betrachten, dass eine stringente Einführung in das Thema relativ schwierig ist. Dieses Dokument behandelt im Wesentlichen die normale Handhabung von *git* als Solo-Developer und im Team. Ein weiteres Dokument kümmert sich dann um fortgeschrittene Themen im Umfeld Git-Server.

Da *git* im Hintergrund auf Linux basiert, werden wir uns auch oberflächlich mit der Arbeit im Terminal beschäftigen müssen.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

1 Installation

1.1 Vorabinformation

Die Installation auf dem lokalen Rechner (=Client) ist auf den üblichen drei Systemen (Windows, Linux, MacOS) relativ problemlos. Wenn du mit dem IPad arbeiten willst, musst du auf eine gehostete Version im Internet zurückgreifen, die browserbasiertes Arbeiten ermöglicht.

Alle Arbeiten in den nächsten Kapiteln finden ausschließlich auf deinem eigenen Gerät statt und sind auch ohne Netzwerkanbindung möglich. Die einzige Ausnahme ist der Download vom Material, das diesen Kurs begleitet.

1.2 Client-Installation

Für die ersten Schritte mit *git* ist es sehr sinnvoll, eine ganz einfache Client-Software (kurz Client) zu verwenden, die ohne grafische Benutzeroberfläche auskommt – du musst als mehr mit der Tastatu als mit der Maus arbeiten!. Ein *einfacher Client* ist übrigens nicht im Sinne von *einfacher Bedienung*, sondern eher im Sinne von *ursprünglicher Version* gemeint. Mittlerweile bringt jede Entwicklungsumgebung bereits eine eigene Version eines *git*-Clients mit und wir können beim besten Willen nicht jede Variante, eventuell noch mit speziellen Features, besprechen.

Faktisch gibt es eigentlich keinen reinen *git*-Client, weil immer automatisch Client und Server installiert werden. Gerade bei Windows bedeutet das aber nicht, dass man den Server auch verwenden kann. Hier hat Microsoft so viele Hindernisse in den Weg gelegt, dass man besser die Finger davon lässt!

1.2.1 Windows

Für die Installation auf Windows ist es hilfreich, wenn du dich an folgende Reihenfolge hältst:

- Notepad++ installieren
  (im Verlauf der Installation wird nach dem bevorzugten Editor gefragt und das einfache *notepad* von Windows ist etwas beschränkt.)
- Git installieren

1.2.1.1 Notepad++

Lade das Programm notepad++ von notepad-plus-plus.org und installiere es durch den üblichen Doppelklick. Es gibt keine Stellen, wo das Vorgehen nicht eindeutig wäre.

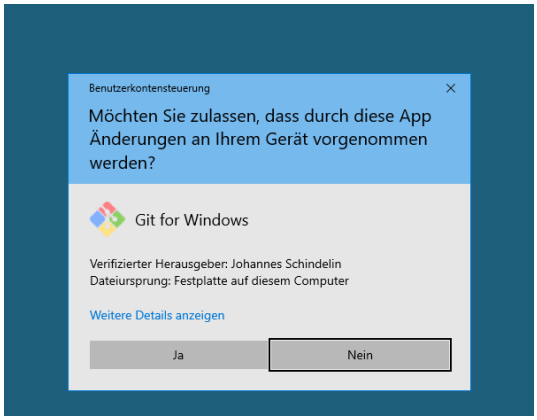
© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

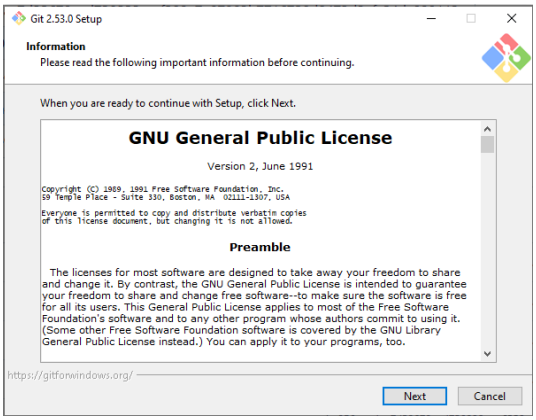
1.2.1.2 Git

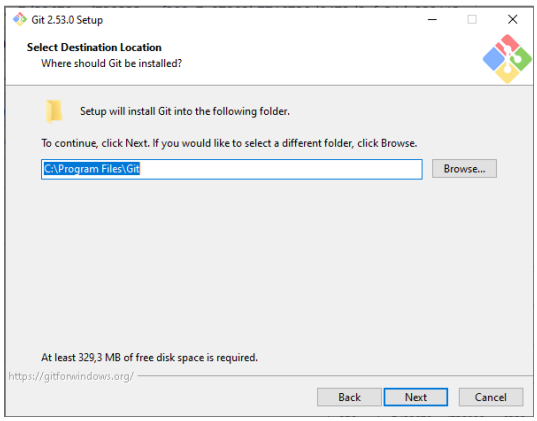
Lade von die Installationsdatei von der Seite gitforwindows.org herunter.

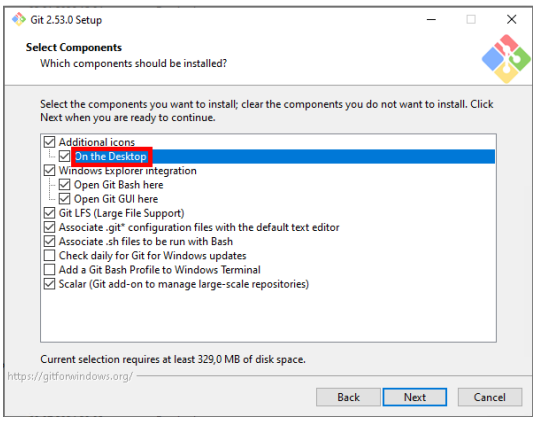
Hinweis
Bei der Installation wirst du gefragt, wie du mit Zeilenumbrüchen verfahren willst. Der Hintergrund ist, dass Windows eine Zeichenkombination „CRLF“ (Zeilenanfang + neue Zeile) wie bei der Schreibmaschine verwendet, wären MacOS und Linux nur *ein einziges* Zeichen „LF“ verwenden. Da *git* im Hintergrund Linux verwendet, muss hier immer umgewandelt werden. *Git* macht das für dich, gibt aber ab und zu eine Warnung aus.

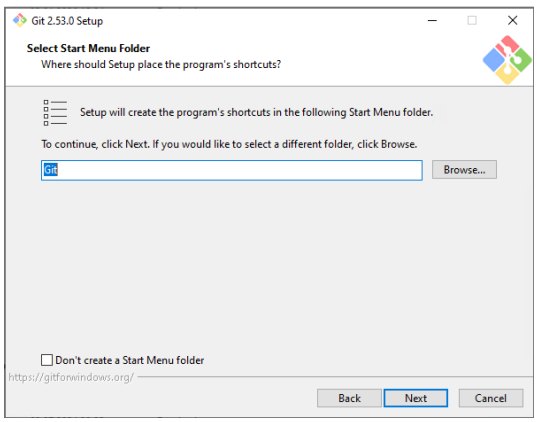
Starte nun die Installation von *git* durch Doppelklick auf die Installationsdatei. Die nachfolgenden Bilder zeigen den Ablauf in der Version vom November 2025. Beachte, dass dieses Handout für die digitale Verwendung gedacht ist – aus diesem Grund füge ich die Bilderstrecke hier ein. Für einen Ausdruck wäre das Verschwendung von Farbe und Papier!

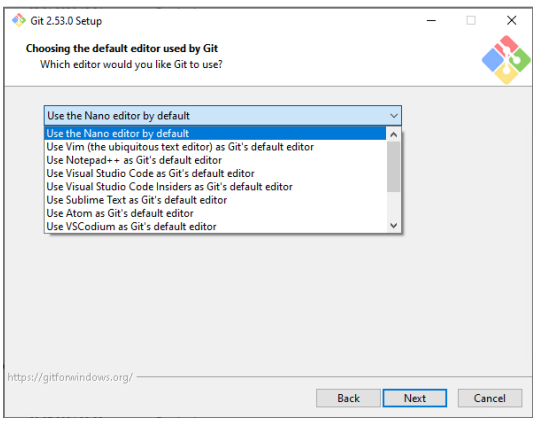




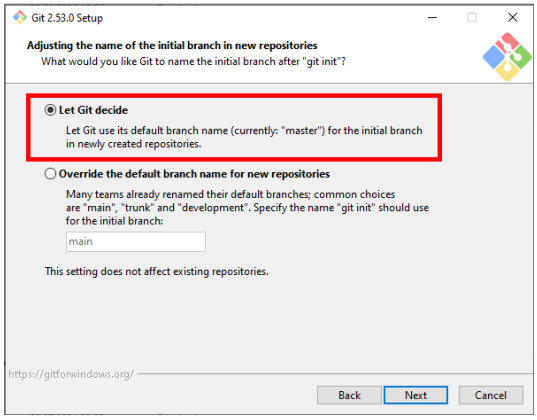


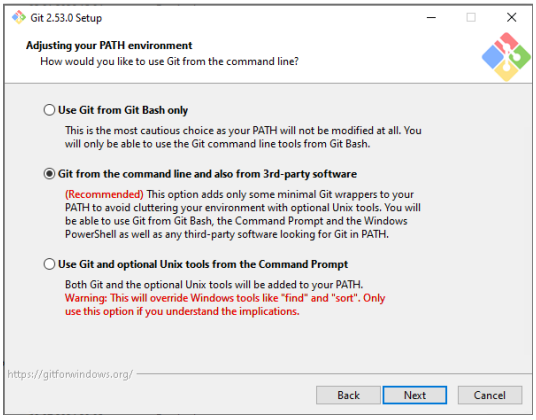


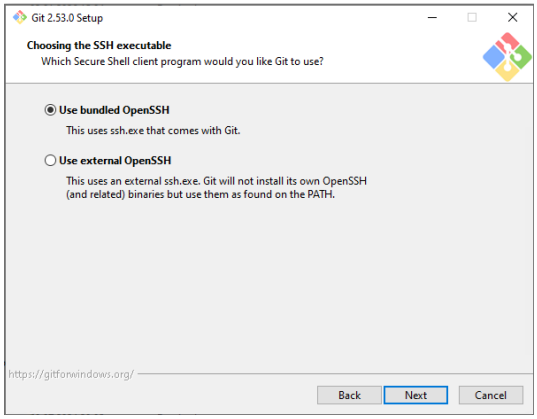


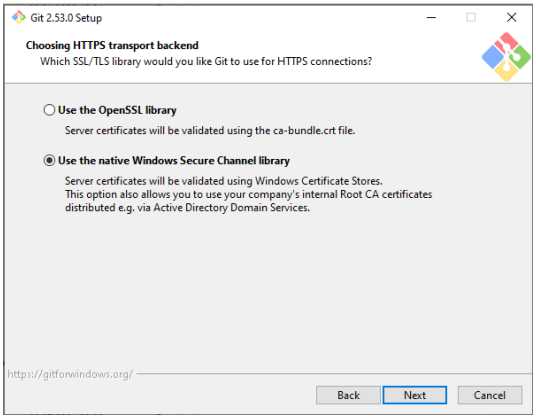


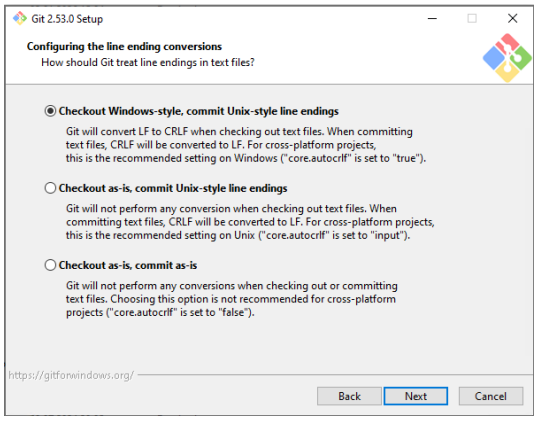
© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

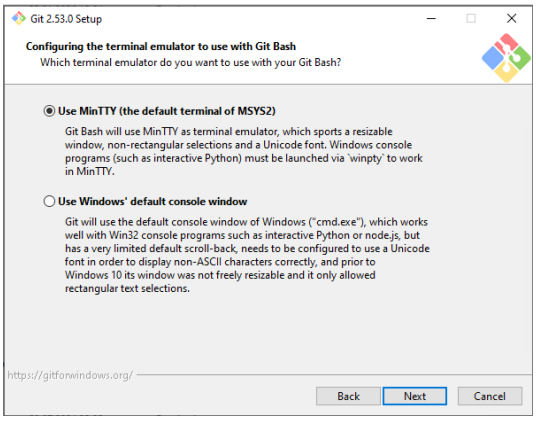


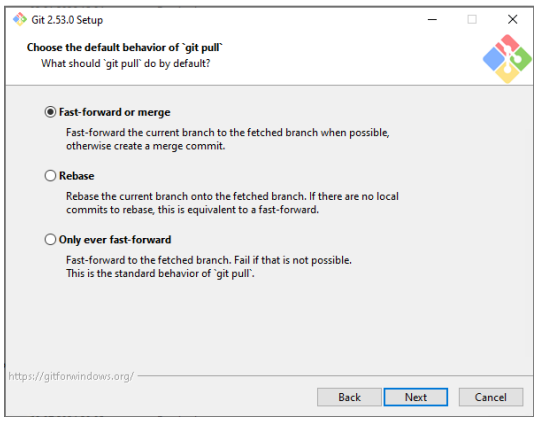


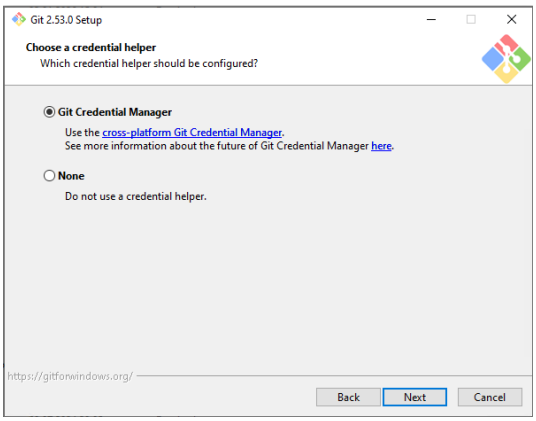




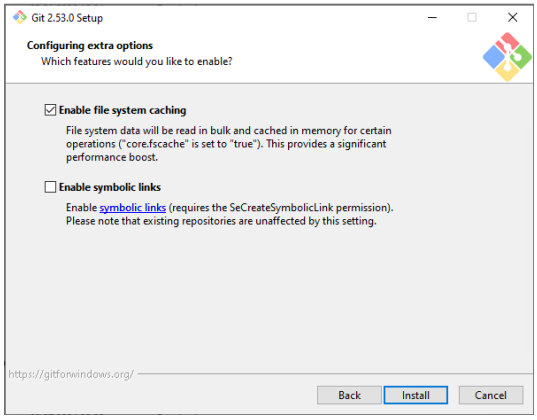


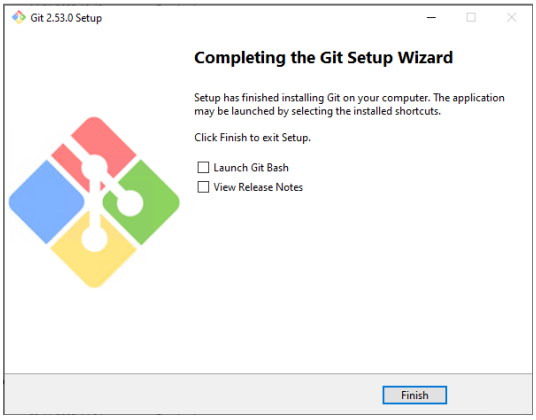






© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.





1.2.2 Linux

Hier hängt es etwas von der Linux-Version (Distribution) ab. Bei Ubuntu genügt zum Beispiel der Befehl

```
apt install git -y
```

1.2.3 MacOS

MacOS ist ähnlich pflegeleicht. Mit `brew install git` ist die Installation ebenfalls bereits gestartet und nach kurzer Zeit beendet.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

2 Lokale Arbeit

2.1 Geplante Inhalte

Das Ziel ist bei der lokalen Arbeit, dass du problemlos verschiedene Versionen von Dateien (zum Beispiel Unterrichtsvorbereitungen) auf deinem Rechner verwalten kannst, ohne bei den Dateinamen zu *kreativ* werden zu müssen. Gleichzeitig kannst du auf diese Weise auch einfach ein Backup deiner Daten erstellen – allerdings erst dann, wenn wir mit einem Server arbeiten[1]

Nachfolgend besprechen wir folgende Punkte

- Grundkonfiguration (einmalig)
- Erstellen eines Repositories
- Versionierung von Dateien
- Einfache Fehlerbehebung

Die hier angegebene Liste von *git*-Befehlen ist keineswegs vollständig, aber mit den markierten Befehlen sind fast alle Aspekte der täglichen Arbeit abgedeckt. Wir werden sie nachfolgend behandeln, wenn auch nicht in genau dieser Reihenfolge.

Befehl	Befehl
git status	`git clean`
git init	**git push**
git log	**git pull**
git add	**git diff**
git commit	`git mergetool`
`git stash`	**git show**
git checkout	**git fetch**
git merge	**git clone**
git branch	**git remote**
git switch	

Darüber hinaus gibt es noch fortgeschrittenere Befehle, die meist der Fehlerbehebung dienen. Auf sie werden wir auch einen Blick werfen, aber nicht zu sehr in die Tiefe gehen.

Befehl	Befehl
`git reset`	`git revert`
`git cherry-pick`	`git restore`
`git rebase`	

[1] Ein Backup auf dem gleichen Rechner ist nur sehr bedint sinnvoll!

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

2.2 Konfigurieren von git

Öffne die *git-Bash*. Auf Linux und MacOS öffnest du ein Terminal. Zukünftig sreche ich pauschal von der *git-Bash*, meine aber immer auch das Terminal der anderen Systeme!

Es ist aktuell egal, in welchem Ordner du dich befindest, wenn du die *git-Bash* öffnest, da es um globale Einstellungen geht. Sollte dir die Schrift im Fenster zu klein sein, kannst du sie mit `strg++` vergrößern.

Aktuelle Einstellungen ansehen

```
# Eingabe
git config --list
```

Was hier genau erscheint, hängt von deinem System ab. Du musst nun deinen Namen und deine Mailadresse an *git* übergeben. Das spielt zwar erst im Team eine Rolle, wird aber als Voraussetzung gefordert!

Einstellungen anpassen

```
# eigene Phantasiedaten verwenden
git config --global user.name "Susi Sandmann"
git config --global user.email "susi@sandmann.de"
```

2.3 Erstellen eines Repositories

Die Voraussetzung für die Arbeit mit *git* ist ein *Projektordner*, in dem deine Projektdateien liegen sollen. In diesem Ordner wirst du dein erstes (lokales) Repository erstellen.

2.3.1 Ordner anlegen

Auch wenn das Erstellen eines Ordners mit den üblichen Werkzeugen des Betriebssystems natürlich möglich ist, werde ich mich dennoch konsequent auf die Arbeit im Terminal beschränken.

```
cd                                  ①
mkdir versuch_1                     ②
cd versuch_1                        ③
```

① Bei Windows landest du damit im Ordner `c:users<DEIN-NAME>`, bei Linux und MacOS bist du in `/home/<DEIN-NAME`

② Den Ordner erstellen (`mkdir = make directory = erstelle Ordner`)

③ Wechsle in den Ordner (`cd = change directory`)

Bist du dir unsicher mit dem Ort des Ordners:

```
pwd
```

Da auch die *git-Bash* eine Form von abgespecktem Linux darstellt, gelten die nachfolgenden Befehle auf allen drei Systemen in gleicher Weise. Es ist sinnvoll, wenn du dir diese Liste bereit legst (oder die Befehle lernst).

Wichtige Befehle

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Befehl	Kurzerklärung
`ls -la`	Auflisten aller Dateien im Ordner
`cd`	Wechsel in den Home-Ordner
`cd <ordner>`	Wechsel in *Ordner*
`cd ..`	Wechsel eine Ebene nach oben
`echo <Text>`	Ausgabe von Text
`> <datei>`	Ausgabe in Datei (s.u.)
`mkdir <ordner>`	Ordner erstellen
`rm -rf <ordner>`	Ordner ohne Rückfrage löschen
`cat <datei>`	Inhalt der Datei komplett ausgeben
`less <datei>`	Langsame Ausgabe, Ende mit `q`

TIPP
Im Terminal kannst du mit der *Pfeil hoch* Taste durch die letzten Befehle blättern – damit gehen manche Dinge sehr schnell!

2.3.2 Anlegen des Repositories

Im Ordner `versuch_1` gibst du nun folgenden Befehl ein[2]:

```
git init
```

Es erscheint eine Meldung, die je nach System etwas anders aussehen kann. Die *git-Bash* macht leider nur englische Ausgaben! Hier musst du selbst übersetzen – die deutschen Texte sind hier für das Verständnis besser.

```
Hinweis: Als Name für den initialen Branch wurde 'master' benutzt.
Hinweis: Dieser Standard-Branchname kann sich ändern. Um den
Hinweis: Namen des initialen Branches zu konfigurieren, der in
Hinweis: allen neuen Repositories verwendet werden soll und
Hinweis: um diese Warnung zu unterdrücken, führen Sie aus:
Hinweis:
Hinweis:    git config --global init.defaultBranch <Name>
Hinweis:
Hinweis: Häufig gewählte Namen statt 'master' sind 'main', 'trunk'
Hinweis:  und 'development'. Der gerade erstellte Branch kann
Hinweis:  mit diesem Befehl umbenannt werden:
Hinweis:
Hinweis:    git branch -m <Name>
Leeres Git-Repository in /tmp/versuch_1/.git/ initialisiert
```

Es hat sich eingebürgert[3], den Arbeitszweig von *master* auf *main* umzubenennen:

```
git branch -m main
```

Den Erfolg kannst du schnell überprüfen:

[2] Er ist in obiger Tabelle nicht markiert, weil man ihn nicht ständig braucht. Wichtig ist er aber trotzdem!
[3] Bedingt durch gesellschaftspolitische Strömungen, die *Master* mit Sklaverei gleichsetzen, aber eine Master-Arbeit hier nicht als Problem sehen.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
git status
```

Es sollte folgender Text (eventuell englisch) erscheinen:

```
Auf Branch main

Noch keine Commits

nichts zu committen (erstellen/kopieren Sie Dateien und benutzen
Sie "git add" zum Versionieren)
```

Hinweis
Wie im Hinweis oben steht, kannst du *git* auch so konfigurieren, dass der Branch eines neu angelegten Repositories automatisch *main* genannt wird.

```
git config --global init.defaultBranch main
```

2.4 Einfacher Workflow

2.4.1 Daten erstellen

Generell ist es (natürlich) möglich, jeden Typ von Datei mit *git* zu versionieren. Eine der Stärken von *git* ist allerdings der Vergleich von Versionsständen auf Zeichenebene und deshalb sind Binärformate wie Musik, Bilder, aber auch Dokumente der gängigen Office-Produkte nur bedingt geeignet. Auch sehr große Dateien sind möglich – aber nicht sinnvoll. Hier kann auf *git*-LFS (Large File Storage) ausgewichen werden, was in diesem Workshop aber nicht Thema ist!

Nun erstellsst du eine erste Datei:

```
echo "Version 1" > datei1.txt
```

Hinweis
Der Befehl `echo` erzeugt eigentlich eine Bildschirmausgabe, die hier in eine Datei umgeleitet wird. Das Symbol > überschreibt den Inhalt der Datei ohne Warnung, mit >> wird der Text an das Ende der Datei angehängt.

2.4.2 Versionieren

Einfaches Versionieren geht sehr schnell mit folgendem Befehl:

```
git add datei1.txt
```

(Varianten dieses Befehls besprechen wir noch!)

Nun schreibsst du einen Eintrag in die Projekt-Historie:

```
git commit -m "Erste Datei erstellt"
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Hinweis
Ein Commit ist ein Projektstand, an den du zurückkehren kannst, wenn etwas grob missglückt ist. Vergeht zu viel Zeit (=viel Code) zwischen zwei Commits, so entstehen eventuell größere Verluste bei einem Rücksprung zum alten Stand.
Später im Workshop lernst du auch, wie du Commits nachträglich zusammenfassen kannst (=squash) um deine Entwicklungsschritte übersichtlich zu halten.

Damit ist der Versionierungsablauf eigentlich schon definiert:

- Dateien, in denen du Änderungen vorgenommen hast, sammelst du mit *add*
- Haben sich genug Daten angesammelt – mit Sinnzusammenhang – dann erstellst du einen *Commit* mit einer aussagekräftigen Beschreibung.

Hinweise
Wenn du die Beschreibung lieber mit dem Editor machen möchtest, rufst du *nur* `git commit` auf (also ohne `-m`).

In welchen Zeitabständen oder Sinnzusammenhängen du Commits erstellst, ist alleine *deine* Entscheidung. Du solltest aber bedenken, dass ein Rücksprung zu einem älteren Commit von *allen* Dateien die Vorgängerversion restauriert. Wenn du also zu viele Änderungen am Code vorgenommen hast, die behalten willst, dann wirst du einiges an Handarbeit haben. Deshalb lieber mehr Commits in kürzeren Abständen!

Ändere deine Datei nun ab, indem du eine Zeile ergänzt:

```
echo "Version 2" >> datei1.txt
git add datei1.txt
git commit -m "Erläuterung"
```

Die Historie besitzt nun zwei Punkte:

```
git log --oneline

# Ausgabe
eaa6289 (HEAD -> main) Erläuterung
afbc9c5 Erste Datei erstellt
```

Die Hash-Werte (Zahlen am Zeilenanfang) können bei dir anders sein!

Hinweis
Du kannst natürlich auch mehrere Dateien in einem Rutsch hinzufügen!

```
git add .   # Alle Dateien im aktuellen Ordner
            # und den Unterordnern

git add --all  # (oder -A) fügt alle Dateien
               # ab der obersten Ebene des
               # Projekts hinzu
```

Wir werden später sehen, was genau beim *add* passiert, zum Beispiel wenn man die gleiche Datei zwischen zwei Commits mehrfach ändert und *addet*.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Zusätzlich kann es natürlich auch sein, dass du bestimmte Dateien überhaupt nicht in die Versionsverwaltung aufnehmen willst. Das ist zum Beispiel dann der Fall, wenn sich Kennwörter und Zugangsdaten in einer Datei befinden. Sobald du die Namen dieser Datei(en) in eine Datei mit dem Namen `.gitignore` einträgst, werden sie beim *add* ignoriert.

```
# Format der Datei
*.log
*.tmp
zugangsdaten.txt
```

Hinweis
git versioniert keine leeren Ordner, weil es in ihnen keine Dateien gibt – *git* arbeitet dateibasiert! Soll ein leerer Ordner versioniert werden, so musst du eine Datei `.gitkeep` im Ordner erstellen.
Der Name ist eine Konvention, keine Verpflichtung. Der führende Punkt *versteckt* die Datei bei den Standardeinstellungen im Explorer (o.ä.).

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

3 Hands On I

3.1 Übung

Besonders bei komplexen Systemen wie *git* gilt: *use it or lose it*!
Du solltest deshalb zur Übung einige Repositories anlegen, um den Ablauf zu verinnerlichen. Beginne jeweils mit

```
cd
```

Auf diese Weise verhinderst du, dass du versehentlich in einem Repository ein *Unter*-Repository erstellst.

3.2 Ein genauerer Blick auf Git

Hinweis
Um die nachfolgenden Erläuterungen praktisch nachvollziehen zu können, verwendest du wieder das Repository `versuch_1`, mit dem wir begonnen haben.

Steht ein Ordner unter der Verwaltung von *git*, so kann man von drei *logischen* Bereichen sprechen:

- *Working-Directory* (Arbeitsordner)
- *Stage* (auch Index oder Cache genannt)
- *Repository*[1]

Das *Working-Directory* enthält die Dateien an denen gearbeitet wird. Es entspricht auf den ersten Blick genau einem normalen Ordner für die Projektentwicklung. Erst die Initialisierung durch *git* erzeugt dann einen weiteren, versteckten Ordner mit dem Namen *.git*. Dieser Ordner bildet **gleichzeitig** den *Stage* und das *Repository*[2].

Hinweis
Es kann sein, dass in deinem Betriebssystem Ordner und Dateien mit einem führenden Punkt nicht angezeigt werden. Dies ist aber im Dateimanager konfigurierbar.

Logische Bereiche bedeutet in diesem Zusammenhang, dass die Bereiche von *git* verwaltet werden und vom unbedarften Benutzer nicht einfach zu unterschieden sind. In einem späteren Abschnitt werden wir einen genaueren Blick in diesen Ordner werfen.

Betrachten wir zunächst einen Entwicklungszyklus im Detail. Dabei wirst du im Wesentlichen die gleichen Befehle wie vorher verwenden, allerdings untersuchen wir einige Varianten, die für das Verständnis wichtig sind.

[1] Je nach Quelle und Kontext wird der Begriff etwas inkonsistent verwendet
[2] Hier siehst du den inkonsistenten Gebrauch von „Repository“. Einerseits wird der gesamte Projektorder als Repository bezeichnet, sobald er von *git* verwaltet wird, andererseits befindet sich das eigentliche Repository im Ordner *.git*.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Du bist immer noch im aktuellen Ordner `versuch_1`. Er sollte aktuell *eine Datei* und *zwei Commits* enthalten.
Das kannst du mit folgenden Befehlen überprüfen:

```
ls        # Anzeige der Dateien
git log   # Was hat git gemacht
```

Neben dem Dateinamen liefert das folgende Ausgabe (andere Hashes!):

```
commit 6db4f3383512a7f3aaaf3b050334d7fd2c474688 (HEAD -> main)
Author: Wolfgang <lehrer@t-online.de>
Date:   Thu Nov 13 10:46:50 2025 +0100

    Erläuterung

commit 430149c83906005ae2c53c8eaf9ce230e3456ad0
Author: Wolfgang <lehrer@t-online.de>
Date:   Thu Nov 13 10:46:50 2025 +0100

    Erste Datei erstellt
```

Ob noch Änderungen offen sind, zeigt dir:

```
# Eingabe
git status

# Ausgabe (Zeilenumbrüche geändert)
Auf Branch main
nichts zu committen,
Arbeitsverzeichnis unverändert
```

Das bedeutet, dass du aktuell einen *sauberen* Zustand deines Arbeitsordners ohne Änderungen an Dateien vorliegen hast.

Erstelle nun eine *neue Datei* `datei2.txt` und füge die Zeile „*Version 1*“ ein.
In der Datei `datei1.txt` kommt die Zeile „*Version 3*“ dazu:

```
# Eingabe (Lasse ich zukünftig weg)
echo "Version 3" >> datei1.txt
echo "Version 1" >> datei2.txt

# Status anzeigen lassen
git status

# Ausgabe
Auf Branch main
Änderungen, die nicht zum Commit vorgemerkt sind:
  (benutzen Sie "git add <Datei>...",
   um die Änderungen zum Commit vorzumerken)

  (benutzen Sie "git restore <Datei>...",
   um die Änderungen im Arbeitsverzeichnis zu verwerfen)

    geändert:       datei1.txt                                  ①
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
Unversionierte Dateien:
  (benutzen Sie "git add <Datei>...",
   um die Änderungen zum Commit vorzumerken)

    datei2.txt                                                  ②

keine Änderungen zum Commit vorgemerkt
(benutzen Sie "git add" und/oder "git commit -a")
```

① Bereits versioniert aber geändert
② Noch nicht in *git* registriert!

Es wird dir angezeigt, dass Änderungen im *Working-Directory* vorliegen.
Du siehst auch, dass *git* die Datei `datei1.txt` bereits kennt (=geändert) und dass `datei2.txt` noch nicht in die Versionsverwaltung aufgenommen wurde (=unversioniert).

Vorsicht
Du findest im Internet oft den Befehl `git commit -am "bemerkung"`.
Hier musst du wissen, dass bei `-am` nur bereits versionierte Dateien in geänderter Form in den Commit übernommen werden – `datei2.txt` würde also ignoriert.

Übertrage nun die Änderungen beider Dateien *auf den Stage/in den Stage*:

```
# Dateien aufnehmen
git add .

# Status anzeigen
git status

# Ausgabe
Auf Branch main
Zum Commit vorgemerkte Änderungen:
  (benutzen Sie "git restore --staged <Datei>..."
   zum Entfernen aus der Staging-Area)

    geändert:       datei1.txt
    neue Datei:     datei2.txt
```

Nun sind beide Dateien zum Commit vorgemerkt, also zum festen Eintrag in die Projekt-Historie – Das führst du jetzt aber **noch nicht aus**!

Als Demonstration fügst du beiden Dateien eine weitere Zeile hinzu und fragst wieder den Status ab:

```
echo "Weitere Zeile" >> datei1.txt
echo "Weitere Zeile" >> datei2.txt
git status

# Ausgabe ( "----"" sind Anmerkungen )
Auf Branch main
Zum Commit vorgemerkte Änderungen:
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
  (benutzen Sie "git restore --staged <Datei>..."
   zum Entfernen aus der Staging-Area)

    geändert:       datei1.txt
    neue Datei:     datei2.txt

--------------- Ende Stage ------------
--------------- Anfang Workdir --------

Änderungen, die nicht zum Commit vorgemerkt sind:
  (benutzen Sie "git add <Datei>...", um die Änderungen
   zum Commit vorzumerken)

  (benutzen Sie "git restore <Datei>...",
   um die Änderungen im Arbeitsverzeichnis zu verwerfen)

  geändert:       datei1.txt
  geändert:       datei2.txt
```

Von `datei1.txt` gibt es nun 3 verschiedene Versionen:

- Version 1 in der Historie (durch den Commit)
- Version 2 im Stage (durch add)
- Version 3 im *Working-Directory* (durch echo)

Bei Datei `datei2.txt` sind es nur nur zwei Versionen und keine davon ist in die Historie aufgenommen.

Der nächste *Commit* transferiert jetzt **nur** die Versionen der Dateien *aus dem Stage* ins Repository, also in die Projekt-Historie. Das führt dann zu folgender Situation:

```
git commit -m "nur Stage"
```

```
git Status

# Ausgabe
Auf Branch main
Änderungen, die nicht zum Commit vorgemerkt sind:
  (benutzen Sie "git add <Datei>...", um die Änderungen zum Commit
  vorzumerken)
  (benutzen Sie "git restore <Datei>...", um die Änderungen im
  Arbeitsverzeichnis zu verwerfen)
    geändert:       datei1.txt
    geändert:       datei2.txt

keine Änderungen zum Commit vorgemerkt (benutzen Sie "git add"
und/oder "git commit -a")
```

Die oberen beiden Dateiversionen sind aus dem Stage verschwunden sind (sind jetzt Teil des Commits) und die aktuellen Änderungen sind nach wie vor im Arbeitsverzeichnis vorhanden, weil für sie noch kein *add* stattgefunden hat.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
# Zum Stage hinzufügen
git add .

git status

# Ausgabe

Auf Branch main
Zum Commit vorgemerkte Änderungen:
  (benutzen Sie "git restore --staged <Datei>..." zum Entfernen aus
  der Staging-Area)
    geändert:       datei1.txt
    geändert:       datei2.txt
```

Hinweis
Wenn ich hier von *transferieren* spreche, so führt das zu einer falschen Vorstellung. In Wirklichkeit werden im *.git*-Ordner nur einige Verweise (Links, Zeiger) geändert – wobei das auch wieder nicht die ganze Wahrheit ist!

3.2.1 Etwas mehr Wahrheit

Dieser Abschnitt ist nur als Einblick für Interessierte gedacht! Bitte installiere dir in der *git-Bash* das Programm *tree*, um den Ausführungen folgen zu können. Die Anleitung befindet sich im Anhang.

Ganz am Anfang musst du wissen, dass es 4 Git-Objekte gibt
(Keine Angst das machen gleich noch etwas genauer!)

1. blob
2. tree
3. commit
4. tag

blob
„Binary large Objects“ sind reine Datei**inhalte**, die *ohne* ihren wirklichen Dateiname oder Metadaten gespeichert sind.

tree
Ein *Tree* ist ein Abbild des Arbeitsverzeichnisses. Er enthält aber nur die Namen, Typen und Hashes der Dateien, Ordner und Unterordner. Die eigentlichen Dateiinhalte (=Blobs) sind getrennt davon gespeichert. Ein Tree ist deshalb klein und stellt keine großen Ansprüche an den Speicherplatz im Versions-System.

commit
Ein *Commit* ist ein *Zeiger* auf den *Tree*, der zu einem bestimmten Zeitpunkt aktuell war. Er enthält noch weitere Metadaten.

tag
Er ist ein Zusatzname für ausgewählte Commits (z.B. Releases), um ohne Kenntnis des Hashs darauf zugreifen zu können.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Hinweis
Erwarte nicht von dir, dass du dir die nachfolgenden Befehle merken kannst – sie sind nicht kompliziert, aber ungewohnt und das ist wie das Lernen von Vokabeln, die man eigentlich nicht braucht. Für das Verständnis sind sie aber wichtig!

3.2.2 Eigene Experimente

Erstelle dir ein neues Repository in einem anderen Ordner

```
cd      # Zurück auf Start ...
git init demo
cd demo
git branch -m main
```

Wenn du nun die Verzeichnisstruktur des Ordners *.git* genauer untersuchst, wirst du in etwa dies hier sehen, weil noch keine Dateien im *Working-Directory* sind. Dafür verwendest du den Befehl

```
tree .git
```

```
.git
├── branches
├── config
├── description
├── HEAD
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── prepare-commit-msg.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── push-to-checkout.sample
│   ├── sendemail-validate.sample
│   └── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
└── refs
    ├── heads
    └── tags
```

Lege nun im leeren *Working-Directory* eine Datei `homepage.html` an[3].

```
echo "Etwas HTML" >> homepage.html
git add homepage.html
```

Über ein erneutes `tree .git` siehst du erste Änderungen im *.git*-Ordner. Im Bild ist aus Platzgründen nur der relevante Ausschnitt angezeigt. Bei dir kann der Hash einen anderen Wert besitzen!

[3] Ich habe absichtlich nicht index.html verwendet, weil „index“ aktuell anders belegt ist.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
│   └── update.sample
├── index <<< NEU
├── info
│   └── exclude
├── objects
│   ├── ce <<< Neu, bei dir evtl. anders
│   │   └── 1332fcf1795166c3c5cd0216b5e7dc2dce7998
│   ├── info
│   └── pack
└── refs
    ├── heads
    └── tags
```

Die Datei `index` stellt den *Stage* dar. Diese Datei kannst du nicht direkt lesen. Das geht nur mit

```
# Lesen der Datei "index"
git ls-files --stage

# Ausgabe
100644 ce1332fcf1795166c3c5cd0216b5e7dc2dce7998 0   homepage.htm
```

Diese Zeile stellt die Verbindung zwischen Dateiname (homepage.html) und dem Datei-Inhalt her, der (s.u.) an anderer Stelle gepeichert ist.

Den Hash aus obiger Ausgabe[4] findet du oben in der Ordnerstruktur wieder.

Allerdings wird mit den ersten beiden Zeichen (ce) ein Ordner erstellt und in diesem Ordner eine Datei, deren Name aus den restlichen Zeichen besteht. *Diese* Datei enthält den Inhalt Daite `homepage.html`. Zusätzlich findest du den Dateityp (100644), den Status (Das ist komplexer! Hier 0) und den Dateinamen.

Den Hash der Datei kannst du dir übrigens auch so anzeigen lassen:

```
git hash-object homepage.html
```

Aber gehen wir einen Schritt weiter und committen die Datei:

```
git commit -m "homepage.html added"
```

Nun sieht die Ordnerstruktur in `.git` verändert aus – die Hooks habe ich in der Darstellung aus Platzgründen entfernt!

[4] In einem richtigen großen Repository erscheint hier eine sehr umfangreiche Liste aller Dateien!

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
│   └── update.sample
├── index
├── info
│   └── exclude
├── logs
│   ├── HEAD
│   └── refs
│       └── heads
│           └── main
├── objects
│   ├── 94
│   │   └── eed25c4ef6cfa1384df66f28308c09dc9bf28d
│   ├── b7
│   │   └── 510dd34252cb3e42bfc82f043d9f42aa00b2f5
│   ├── ce
│   │   └── 1332fcf1795166c3c5cd0216b5e7dc2dce7998
│   ├── info
│   └── pack
└── refs
    ├── heads
    │   └── main
    └── tags
```

Ab hier bekommst du sicher andere, eigene Hash-Werte! Du musst in den nachfolgenden Befehlen also deine eigenen Werte verwenden.

Wir konzentrieren uns auf die *objects* und das *log*.

Mit `git log` erfährst du etwas über den erfolgten Commit – zum Beispiel seinen Hash:

```
commit b7510dd34252cb3e42bfc82f043d9f42aa00b2f5 (HEAD -> main)
Author: wolfgang <lehrer@t-online.de>
Date:   Sun Jan 26 20:20:10 2025 +0100

    homepage.html added
```

Etwas kompakter wird die Darstellung mit `git log --oneline`.

```
b7510dd  (HEAD -> main)  homepage.html added
```

Mit dem Hash `b751...` des Commits wurde wieder ein Ordner `b7` und eine Datei `510dd34...` erstellt.

Du kannst auch (hier unnötig) aus dem Hash ermitteln, welchen Typ von Objekt er darstellt:

```
git cat-file -t b7510dd34252cb3e42bfc82f043d9f42aa00b2f5

# Ausgabe
commit
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Änderst du den Parameter von *-t* (=Typ) auf *-p* (=pretty print = Inhalt), so bekommst du weitere Details:

```
# Eingabe
git cat-file -p b7510dd34252cb3e42bfc82f043d9f42aa00b2f5

# Ausgabe
tree 94eed25c4ef6cfa1384df66f28308c09dc9bf28d   << wichtig!
author wolfgang <lehrer@t-online.de> 1737919210 +0100
committer wolfgang <lehrer@t-online.de> 1737919210 +0100

homepage.html added
```

Der Commit verweist also auf den *tree* mit dem Hash 94ee . . .
Der nächste Befehl liest den Tree aus. Da wir aktuell nur eine einzige Datei haben, ist die Ausgabe sehr kurz,:

```
# Eingabe
git cat-file -p 94eed25c4ef6cfa1384df66f28308c09dc9bf28d

# Ausgabe - normal deutlich mehr Einträge
100644 blob ce1332fcf1795166c3c5cd0216b5e7dc2dce7998 homepage.html
```

Der Eintrag im Tree zeigt uns die Verbindung zwischen Dateiname und eigentlichem Dateiinhalt (ähnlich wie im Index!).

Abschließend sehen wir uns den Inhalt des Blob an

```
# Eingabe
git cat-file -p ce1332fcf1795166c3c5cd0216b5e7dc2dce7998

# Ausgabe
Etwas HTML
```

Du bist jetzt über mehrere Stufen zum Inhalt der Datei gelangt. Eine kleine Stufe haben wir aber ausgelassen:

```
git cat-file -t ce1332fcf1795166c3c5cd0216b5e7dc2dce7998

# Ausgabe
blob
```

Hier nochmal die Zusammenfassung der Funktionsweise:

- Der Commit zeigt auf einen Tree
- Im Tree stehen Dateinamen mit den Hashes
  Das sind nur die Namen und nicht die Dateien!
- Der Hash gehört zu einem *blob* (oder tree),
  also den reinen *Roh*daten der Datei **ohne** Dateinamen

Git verwendet die Trennung von Dateiname und Inhalt, um von verschiedenen Stellen aus auf die gleichen Rohdaten verweisen zu können, ohne weiteren Speicher zu verbrauchen. Ändern sich Dateien zwischen zwei Commits *nicht* (Hash bleibt gleich), so werden sie nicht erneut gespeichert. Der *neue* Commit zeigt auf einen *neuen* Tree in dem wieder die gleichen Hashes mit den gleichen Dateinamen stehen.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Tiefer will ich an dieser Stelle in dieses Thema nicht einsteigen!

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

4 Branches

Am Anfang ist es sinnvoll, wnn du das Bild eines Baumes im Kopf hat, bei dem sich der Stamm (=main Branch) an verschiedenen Stellen in Äste verzweigt. Was in einem solchen Ast passiert, hat keinen Bezug zum Stamm und man kann Äste abschneiden (biologisch nur begrenzt richtig) ohne dass der Stamm Schaden nimmt.

Software-Projekte besitzen immer einen *main*-Branch und (mindestens) einen parallel dazu verlaufenden Entwichlungsbranch.

Für mich selbst sind Branches ganz wichtig, wenn ich Dinge ausprobieren will – gerade bei Themen, wo ich nicht so fit bin. Dann nehme ich ein vorhandenes Projekt (dann muss ich weniger Boilerplate erzeugen) und erstelle beim aktuellen Stand eine Abzweigung mit dem Namen *Labor* oder *Spielplatz* und probiere Dinge aus. Wenn ich scheitere (keine Ausnahme), dann lösche ich diesen Branch einfach wieder und es ist kein Schaden entstanden.

Branches sind in Projekten aber auch wichtig, um die Entwicklungen verschiedener Mitarbeiter voneinander zu trennen – sei es auf dem Server (dazu später mehr) oder auf den individuellen Entwicklungsrechnern.

Im nachfolgenden *Hands On* werden wir uns das etwas genauer ansehen, hier kommen zunächst die Grundlagen.

4.1 Erstellen von Branches

Im Internet findest du viele Seiten, die noch die alten Befehle von *git* verwenden. Sie funktionieren immer noch, ich würde dir aber folgende Variante empfehlen:

```
git switch -c <branch_name>
```

Es ist wichtig zu verstehen, was ein Branch ist und vor allem was er *nicht* ist! Dafür machen wir einige kleine Codebeispiele, die unabhängig von den bisherigen Experimenten sind. Wie üblich richtest du dir zunächst ein Repository ein:

```
cd        # auf Start zurück
git init branch_lab
cd branch_lab
git branch -m main
```

Für einen ersten Commit erstellen wir eine (sinnfreie) Datei:

```
echo "Zeile 1" > datei.txt
git add .
git commit -m "Datei 1 erstellt"
```

Das Log zeigt dann:

```
git log

# Ausgabe
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
commit 70eaef36a87aeb8a1e2eba93577a01329aff34d8 (HEAD -> main)
Author: Wolfgang <lehrer@t-online.de>
Date:   Fri Jan 23 12:57:41 2026 +0100

    Datei 1 erstellt
```

In der ersten Zeile sehen wir, dass es um den Branch `main` geht und dass es einen `HEAD` gibt, der auf `main` zeigt. Das ist wichtig, denn *zeigen* ist genau das benötigte Verb. Technisch ist das eine zweistufige Verlinkung:

- In der Datei `HEAD` steht
- In der Datei
  steht der Hash das aktuellen Commit.

```
tree .git

# Ausgabe
.git
├── branches
├── COMMIT_EDITMSG
├── config
├── description
├── HEAD    <<<< DAS zeigt
├── hooks
│   ├── ...
├── index
├── info
│   └── exclude
├── logs
│   ├── HEAD
│   └── refs
│       └── heads
│           └── main
├── objects
│   ├── 4d
│   │   └── 5819dc3711611c3e6a16b1ed362a6952e980fc
│   ├── 70
│   │   └── eaef36a87aeb8a1e2eba93577a01329aff34d8
│   ├── b1
│   │   └── 367440191d3abc86bb46f955d140fec7eef42a
│   ├── info
│   └── pack
└── refs
    ├── heads
    │   └── main   <<<<< hierher
    └── tags
```

Kurze Überprüfung direkt mit den Dateien:

Inhalt von `HEAD`

```
cat .git/HEAD

# Ausgabe
refs/heads/main
```

Inhalt von `defs/heads/main`

```
cat .git/refs/heads/main

# Ausgabe
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

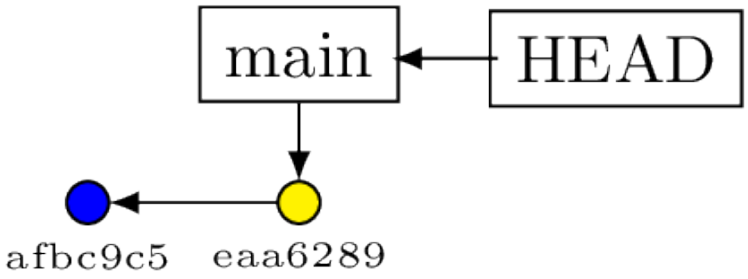
```
70eaef36a87aeb8a1e2eba93577a01329aff34d8
```

Kommen weitere Commits hinzu, so ändert sich der Inhalt von `HEAD` nicht, aber der Inhalt der `.git/refs/heads/main` wird aktualisiert.

```
echo "Zeile 2" >> datei.txt
git add . && git commit -m "Zeile 2"

echo "Zeile 3" >> datei.txt
git add . && git commit -m "Zeile 3"

git log --oneline

# Ausgabe
7c5a585 (HEAD -> main) Zeile 3
1d4966e Zeile 2
70eaef3 Datei 1 erstellt
```

In der Datei steht nun:

```
cat .git/refs/heads/main

# Ausgabe
7c5a5851fcb062c07a9b99195e33b97ee5353ab0
```



Der Pointer *main* wird also von Commit zu Commit geschoben und der Pointer *HEAD* zeigt ständig auf *main*.

Wenn wir nun einen Branch erstellen und auch gleich in diesen wechseln – das macht `git switch -c`, dann könne wir das auch in den Dateien nachverfolgen:

```
git switch -c spielplatz
git log

# Ausgabe
Zu neuem Branch 'spielplatz' gewechselt
commit 7c5a585...gekürzt (HEAD -> spielplatz, main)
Author: Wolfgang <lehrer@t-online.de>
Date:   Fri Jan 23 13:38:07 2026 +0100
```

Die erste Zeile gibt an, dass es zwei Branches gibt, die auf gleichem Stand sind (spielplatz und main). HEAD zeigt auf spielplatz, d.h. in der Datei `Head` steht nicht mehr *ref: refs/heads/main*, sondern

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
cat .git/HEAD

#Ausgabe
ref: refs/heads/spielplatz
```

Da *spielplatz* und *main* aber auf gleichem Stand – dem aktuellen Commit – sind, erhalten wir als Inhalt der Dateien den gleichen Hash:

```
cat .git/refs/heads/spielplatz
7c5a5851fcb062c07a9b99195e33b97ee5353ab0

cat .git/refs/heads/main
7c5a5851fcb062c07a9b99195e33b97ee5353ab0
```

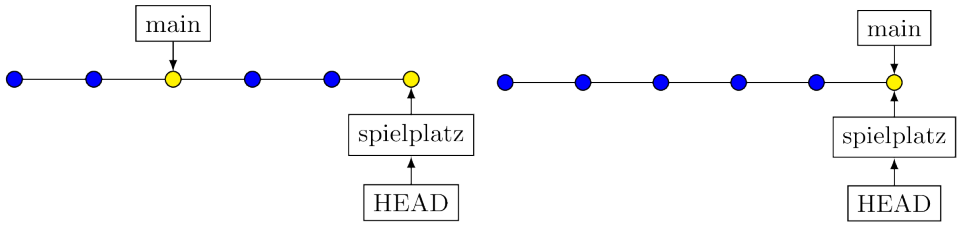
Erst wenn im Branch `spielplatz` neue Commits entstehen, wird der Inhalt der Datei `.git/refs/heads/spielplatz` entsprechend mit dem aktualisierten Hash gefüllt.

Ein neuerlicher Wechsel des Branches zurück auf `main` bewirkt dann also nur noch eine Änderung der Verlinkung in der Datei `.git/HEAD`.

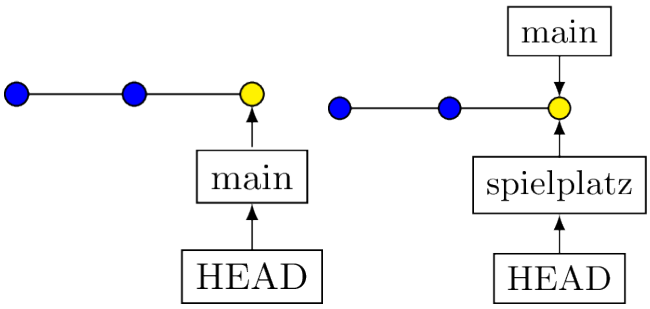
4.2 Zusammenführen von Branches (merge)

Dazu kommen weiter hinten im Script noch Varianten, deshalb möchte ich hier nur die Grundlagen thematisieren. *Merges* dienen dem Abgleich verschiedener Branches. Das ist nötig, wenn Änderungen am Code nicht nur in einem Branch verfügbar sein dürfen.

Der unkomplizierteste Fall ist es, wenn du in einen Branch wechselst, dort arbeitest und danach das Ergebnis direkt wieder in den ursprünglichen Branch übernimmst. Dieser Vorgang wird *fast Forward (ff)* genannt, weil hier eigentlich gar nichts passiert, außer dass die Pointer passend umgeschrieben werden.



Eine typische Fehlvorstellung ist es, dass beim *Branching* sofort eine Abzweigung erzeugt wird. Korrekt ist aber, dass einfach nur ein Zeiger (also eine Datei, die einen Hash enthält) erzeugt wird, der auf den aktuellen Commit zeigt.



© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Der Befehl zur Zusammenführung wird vom Zielbranch aus gegeben. Du **holst** dir also einen Branch an deinen aktuellen Ort:

```
git switch main
git merge spielplatz
```

Komplizierter werden die Dinge, wenn in beiden Branches Änderungen an den Dateien vorgenommen werden – dazu aber etwas später.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

5 Fehlerbehebung

Ein Versionsverwaltungsprogramm hat unter anderem die Aufgabe, ein Projekt in gewissem Umfang *revisionssicher* zu protokollieren. Aus diesem Grund müssen bestimmte *Fehler* oder *dokumentierte Probleme* in der History erhalten bleiben, auch wenn sie später behoben wurden.

Allerdings treten auch bei der Bedienung von *git* Fehler auf – im Sinne von frühem Adden oder Committen, …
Derartige Fehler müssen möglichst auf dem eigenen Rechner repariert werden, *bevor* die Dateien dem Team zur Verfügung gestellt werden. Nachfolgend besprechen wir einige Szenarien mit kleinen Beispielen. Aktuell ist das Arbeiten mit verschiedenen Branches noch nicht Thema und auch Teamarbeit lassen wir außen vor. Wenn wir die aktuellen Beispiele behandelt haben, ist der Schritt zur Teamarbeit nur sehr kurz.

Ein Hinweis vorab
Im Netz findet man statt `restore` oft noch die Variante mit `checkout`, die früher üblich war.

5.1 Kleine Fehler 1

Dieser Fall liegt vor, wenn beim Programmieren in einer Datei ein Fehler entsteht, dessen manuelle Behebung zu aufwändig ist. Die einfachere Lösung ist die Wiederherstellung der betroffenen Datei in der letzten lauffähigen Version. Solange kein *add* erfolgt ist, kannst du die Datei aus dem Stage (=Stand vom letzten Add bzw. Commit) wieder in das *Working-Directory* kopieren (siehe Hinweis unter dem Code!).

```
cd
rm -rf beispiel_1

git init beispiel_1
cd beispiel_1
git branch -m main

echo "Alles gut" > datei.txt
git add .
git commit -m "alles gut"

# Hier kommt der Fehler
echo "Mist gebaut" >> datei.txt

# Stop von dem Restore
echo "Hier mit strg+c abbrechen"
read

# Alte Version vom Stage wieder herstellen
git restore datei.txt
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Hinweis
Vorher habe ich geschrieben, dass bei einem Commit die Dateien aus dem Stage in die Projekt-Historie übertragen und aus dem *Stage* entfernt werden. Intern funktioniert das allerdings etwas anders. Die Datei *index* enthält die Namen und Hashes des aktuellen Commits (tree). Mit einem *add* werden Dateinamen ergänzt. Beim Commit wird die Datei nicht geleert, die Hashes bleiben also verfügbar. Dass `git status` diese Dateien nicht mehr anzeigt, liegt einfach daran, dass der Befehl eigentlich nur ein *diff* zwischen *index* und aktuellem *tree* ausführt.

5.2 Kleiner Fehler 2

In diesem Fall hast du durch ein voreiliges `git add` den Fehler bereits in den *Stage* geschoben. Ein `git restore` wie oben geht nun nicht mehr! Du musst also einen weiteren Schritt rückwärts gehen.

```
cd
rm -rf beispiel_2

git init beispiel_2
cd beispiel_2
git branch -m main

echo "Alles gut" > datei.txt
git add .
git commit -m "alles gut"

# Hier kommt der Fehler
echo "Mist gebaut" >> datei.txt

git add .   # DAS ist anders!

# GEHT NICHT MEHR
git restore datei.txt

#-------- LÖSUNG

echo "git log => HASH"
echo "git restore --source=<HASH> datei.txt"

#--------- ALTERNATIV
echo "git show <HASH>:datei.txt > datei_alt.txt"
```

Du musst also mit `git log` den Hash vom letzten Commit ermitteln und dann geht

```
git restore --source=<HASH> datei.txt
```

Falls du die Datei schon längere Zeit bearbeitet hast und die Änderungen immer mit `add` auf den Stage geschrieben hast – ohne Commits – dann wirst du natürlich Verluste haben! Hättest du kleinschrittig committet, so könntest du auf verschieden Stände zurücksetzen.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Zusatz
Mit `restore` können natürlich nicht nur Dateien aus dem direkt vorausgehenden Commit entnommen werden! Kennst du den Hash, so kannst du auf jeden Commit zugreifen. Allerdings wird dann immer die aktuelle Datei dieses Namens überschrieben! Eine Alternative *ohne* dieses Problem sieht so aus:

```
git show <HASH>:datei.txt > datei_alt.txt
```

5.3 Größere Fehler

Du hast nach dem letzten Commit viele Fehler in einer Vielzahl von Dateien erzeugt und willst deshalb den aktuellen Stand verwerfen. Hier hilft `reset`. Diesen Befehl gibt es in 3 Versionen

- hard
- mixed
- soft

Im Material sind für diese Szenarien passende Scripts hinterlegt, damit du nicht so viel tippen musst.

5.3.1 Hard

```
cd
rm -rf beispiel_3

git init beispiel_3
cd beispiel_3
git branch -m main

# alles funktioniert
echo "Alles gut datei1" > datei1.txt
echo "Alles gut datei2" > datei2.txt
# ... viele Dateien
git add .  && git commit -m "alles gut"

# Hier kommen DIE Fehler
echo "Mist gebaut" >> datei1.txt
echo "Mist gebaut" >> datei2.txt
# ... viele Fehler in vielen Dateien

echo "Hier STRG+c um den Stand analysieren zu können"
read

# Rücksetzen des kompletten Projekts auf den letzten Commit
git reset --hard HEAD
```

Analyse
Die Dateien `datei1.txt` und `datei2.txt` haben folgende Hashes

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
git hash-object datei1.txt datei2.txt
7b67032c95794ec14b091904cfb2a011e39cc073
ee6ebcee0a9dc07e6e7e9b9e4e5d0a044d79e413
```

und enthalten beide zwei Zeilen:

```
cat cat datei1.txt datei2.txt

# Ausgabe
Alles gut datei1
Mist gebaut
Alles gut datei2
Mist gebaut
```

Im Stage befinden sich die Versionen mit den folgenden Hashes:

```
git ls-files --stage
100644 bdea69f7adae8e49efd5e830f280d0fb8369ece7 0   datei1.txt
100644 aca0aa026a74a22559b5a4710e17a3f111da9588 0   datei2.txt
```

Sie enhalten jeweils nur eine einzige Zeile:

```
git cat-file -p bdea69f7adae8e49efd5e830f280d0fb8369ece7
# Ausgabe: Alles gut datei1

git cat-file -p aca0aa026a74a22559b5a4710e17a3f111da9588
#Ausgabe: Alles gut datei2
```

Nun führen wir einen *Hard* Reset aus:

```
git reset --hard HEAD
```

Betrachten wir nun die Hashes in *Stage* und work:

```
git ls-files --stage
# 100644 bdea69f7adae8e49efd5e830f280d0fb8369ece7 0 datei1.txt
# 100644 aca0aa026a74a22559b5a4710e17a3f111da9588 0 datei2.txt

git hash-object datei1.txt datei2.txt
# bdea69f7adae8e49efd5e830f280d0fb8369ece7
#aca0aa026a74a22559b5a4710e17a3f111da9588
```

Die Hashes sind identisch. Hiermit ist also alles gelöscht, was du nach diesem Commit produziert hast – unwiederruflich weg!

Hinweis
Du kannst in der Commit-History auch weiter zurück springen und somit die letzten Commits verwerfen. Das ist völlig problemlos, solange du alleine, lokal auf deinem Rechner arbeitest. Im Team oder auch nur mit Serverabgleich wird das extrem problematisch!

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

5.3.2 Mixed

Das ist so ziemlich die gebräuchlichste Variante. Auch hierzu gibt es ein Script im Material:

```
#!/bin/bash

# Repository löschen falls vorhanden
cd
rm -rf demo_mix
git init demo_mix
cd demo_mix
git branch -m main

for i in {1..10}; do
  echo "Zeile $i" >> datei.txt
  git add . && git commit -m "Zeile $i eingefügt"
  git tag commit_$i
done

cat datei.txt # liefert 10 Zeilen

git reset --mixed commit_4

cat datei.txt # liefert immer noch 10 Zeilen

git log --oneline # liefert 4 Zeilen

git status # zeigt eine geänderte, versionierte Datei
```

Das Script erzeugt eine Datei mit 10 Zeilen:

```
Zeile 1
Zeile 2
...
Zeile 10
```

Sammeln wir die Hashes:

```
# stage
100644 30bbed864cbbe610d59273c86c1ae398acdb9799 0   datei.txt

# workdir
git hash-object datei.txt
# 30bbed864cbbe610d59273c86c1ae398acdb9799
```

Nun machen wir einen Reset auf Commit 4 und sehen uns den Stand genau an:

```
git reset --mixed commit_4

# Ausgabe
Nicht zum Commit vorgemerkte Änderungen nach Zurücksetzung:
M        datei.txt
```

Was sagen die Hashes?

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
# stage
git ls-files --stage
# 100644 01cfd9be0ee0c14e4547ef3182205466edb327cd 0 datei.txt

# workdir
git hash-object datei.txt
# 30bbed864cbbe610d59273c86c1ae398acdb9799
```

Das sind verschiedene Dateien – welche enthält was?
Sehen wir uns die Datei auf dem *Stage* an:

```
git show 01cfd9be0ee0c14e4547ef3182205466edb327cd

# Ausgabe
Zeile 1
Zeile 2
Zeile 3
Zeile 4
```

Das passt zu Commit 4 – so weit so gut. Was ist mit der Datei im *Working-Directory*?

```
cat datei.txt

# Ausgabe
Zeile 1
Zeile 2
...
Zeile 9
Zeile 10
```

Der letzte Stand des *Working-Directory* vor dem Reset wurde also nicht verändert, die Zwischen-Commits 5-10 wurden aber gelöscht! Du kannst nun auf dem letzten Stand weiterarbeiten und neue Commits erstellen. Auch hier ist natürlich wichtig, dass die Commits vorher deinen Rechner noch nicht verlassen hatten!

Hinweis
Immer wenn *gelöscht* verwendet wird, bedeutet das zunächst *ausgeblendet*. Wirklich verschwinden werden deratige Löschungen erst nach 90 Tagen. Mit `git reflog` kannst du deine Aktionen nachvollziehen und gezielt wieder auf einen der gelisteten Commits zurücksetzen - z.B. mit `git reset`.

```
git reset --mixed <HASH>
# gleichwertig:  git reset <HASH>
```

5.3.3 Soft

Verwende wieder das gleiche Script wie eben, ersetze aber `--mixed` durch `--soft`. Auf den ersten Blick nichts anderes als bei `--mixed`. Die Datei `datei.txt` enthält wieder alle 10 Zeilen – das workdir hat sich also wieder nicht verändert. Damit muss also im *Stage* etaw anderes passiert sein. Ein Vergleich der Hashes zeigt uns:

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
git ls-files --stage
# 100644 30bbed864cbbe610d59273c86c1ae398acdb9799 0 datei.txt
git hash-object datei.txt
# 30bbed864cbbe610d59273c86c1ae398acdb9799
```

Stage und *Working-Directory* sind also auf dem gleichen Stand.

5.4 Checkout

Willst du auf einen früheren Commit wechseln um einen früheren Stand des Projekts zu untersuchen, dann ist ein *reset* nicht geeignet. Hier verwendest du einen `checkout`.

```
cd
rm -rf suche

git init suche
cd suche
git branch -m main

for i in {1..5};do
 echo "Version $i" > datei.txt
 git add .
 git commit -m "Version $i"
 git tag "V${i}"
done

echo "gewünschte Version" > datei.txt
git add .
git commit -m "Version 6"
git tag "V6"

for i in {7..10};do
 echo "Version $i" > datei.txt
 git add .
 git commit -m "Version $i"
 git tag "V${i}"
done
```

Du erkennst aus dem Script, dass Version 6 die Datei in der gewünschten Form enthält. Im Ernstfall weißt du das wahrscheinlich nicht. Du beginnst also mit der Suche[1]:

```
git log --oneline

# Ausgabe
b55b329 (HEAD -> main) Version 10
0a20f73 (HEAD -> main, tag: V10) Version 10
287e4e9 (tag: V9) Version 9
9c818b5 (tag: V8) Version 8
71b84f5 (tag: V7) Version 7
e7a5510 (tag: V6) Version 6
d796ed3 (tag: V5) Version 5
```

[1] Natürlich gibt es auch Ansätze mit `git show`

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
2fd4ee2 (tag: V4) Version 4
4c5881c (tag: V3) Version 3
8cc1a7c (tag: V2) Version 2
df10857 (tag: V1) Version 1
```

```
git checkout V6
```

Nun kommt eine große Warnung, die dich darüber informiert, dass du gerade außen an der Versionsverwaltung entlang krabbelst. Da du noch nicht mit Branches arbeitest, solltest du aktuell keinerlei Änderungen an den Dateien vornehmen! In die Dateien hinein sehen ist keine Änderung – das ist also ok. Du kannst auch Teile der Inhalte wegkopieren (in einen anderen ordner) um sie später weiterverwenden zu können.

```
Hinweis: Wechsle zu 'V6'.

Sie befinden sich im Zustand eines 'losgelösten HEAD'.
Sie können sich umschauen, experimentelle Änderungen
vornehmen und diese committen, und Sie können alle möglichen
Commits, die Sie in diesem Zustand machen, ohne Auswirkungen
auf irgendeinen Branch verwerfen, indem Sie zu einem
anderen Branch wechseln.

Wenn Sie einen neuen Branch erstellen möchten, um Ihre
erstellten Commits zu behalten, können Sie das (jetzt
oder später) durch Nutzung von 'switch' mit der Option -c
tun. Beispiel:

  git switch -c <neuer-Branchname>

Oder um diese Operation rückgängig zu machen:
  git switch -

Sie können diesen Hinweis ausschalten, indem Sie die
Konfigurationsvariable 'advice.detachedHead' auf 'false'
setzen.

HEAD ist jetzt bei e7a5510 Version 6
```

Hast du die relevanten Zeilen kopiert, so kannst du mit folgendem Befehl wieder in die Versionsverwaltung einsteigen:

```
git checkout main
```

5.5 Sonstiges

Die Fehlerbehebung kann auch das Löschen ganzer Entwicklungs- oder Testzweige (Branches) bedeuten, was wir später noch besprechen.

Trick
Manchmal soll auch nur die Version gelöscht werden, die irrtümlich auf den Stage gescho-

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

ben wurde. Dabei darf das *Working-Directory* nicht verändert werden. Das erreichst du mit `git restore <dateiname> --staged`.
Das brauchst du eigentlich nur, wenn diese Datei doch nicht im nächsten Commit enthalten sein soll.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

6 Hands on II

Du wirst in diesem Abschnitt einige Dinge als redundant empfinden, weil sie auch bereits früher verwendet wurden. Sieh es einfach als Übung und da du dieses PDF ohnehin nicht ausdrucken wirst, ist die minimal erhöhte Seitenzahl auch kein Problem.

Dieses Szenario ist etwas komplexer, da wir mit mehreren Dateien arbeiten und auch parallele Ideen verfolgen wollen (branching).
Da richtiges Programmieren zu lange dauert, simulieren wir die Arbeit eines Autors, der an mehreren Kurzgeschichten arbeitet. Die Geschichten wurden von einer KI generiert um den Arbeitsaufwand zu minimieren.

Um diese Übung nachvollziehen kannst musst du:

- Vorlage auf deinen Computer *clonen*.
  Das ist ein notwendiger Vorgriff auf den Abschnitt der Teamarbeit, den du hier einfach unhinterfragt ausführst.
- Anlegen eines eigenen Repositories
- Öffnen von 2 Fenstern der *git-Bash*. Eines für den Vorlagenordner uns eines für den Buch-Ordner. Du musst ständig zwischen ihnen hin und her wechseln. Wenn dein Bildschirm groß genug ist, solltest du die Fenster nebeneinander legen.

```
mkdir fortbildung
cd fortbildung
git clone https://github.com/gischupa/dillingen_storyvorlage.git vorlage
```

Es entsteht ein Ordner *vorlage*, in dem die verschiedenen Stände des Projektes enthalten sind.

Lege nun dein Repository an:

```
# Eingaben
mkdir buch
cd buch
git init
git branch -m main
```

Im Ordner \ordner{fortbildungen sollten nun die Repositories `vorlage*` und `buch` vorliegen.

6.1 Teil eins von Story 1

Wechsle in das Fenster mit dem **Vorlagenordner** und Checke *Step_1* aus.

```
git checkout Step_1
```

Kopiere die Datei `story1.md` in den Ordner `buch`.

Diese Datei enthält (absichtlich) nur einen Teil der Geschichte! Das Kopieren kannst du auch gerne mit der Maus im Explorer machen, ich beschreibe aber den schnelleren Weg über das Terminal:

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
# aktueller Ort ist der vorlagen-Ordner
cp story1.md  ../buch/
```

Wechsle in das Buch-Fenster

```
git status

# Ausgabe
Auf Branch main                                                    ①

Noch keine Commits

Unversionierte Dateien:
  (benutzen Sie "git add <Datei>...",
   um die Änderungen zum Commit vorzumerken)
       story1.md                                                   ②

nichts zum Commit vorgemerkt, aber es gibt unversionierte Dateien
(benutzen Sie "git add" zum Versionieren)                          ③
```

① Geänderter Branch-Name ist ok.
② Diese Datei ist betroffen.
③ Empfohlener Befehl `git add story1.md`

Übertragen in den Stage

```
# Eingabe
git add story1.md
git status

# Ausgabe
Auf Branch main

Noch keine Commits

Zum Commit vorgemerkte Änderungen:                                 ①
  (benutzen Sie "git rm --cached <Datei>..."
   zum Entfernen aus der Staging-Area)
    neue Datei:
      story1.md                                                    ②
```

① Die Dateien für den nächsten Commit
② Hier nur diese eine Datei.

Der Hinweis `git rm --cached <Datei>` bewirkt nicht das Löschen der Datei aus dem Arbeitsverzeichnis, sondern nur aus dem Stage!

6.2 Teil zwei von Story 1

Der erste Teil von `story1.md` ist nun in die Versionierung aufgenommen, es wurde aber noch kein protokollierter Punkt in der Projekt-History (=commit) erstellt.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Du „schreibst“ an deiner Geschichte weiter und damit ändert sich der Stand zwischen Arbeitsverzeichnis und Stage. Wir schreiben nicht – wir kopieren!
Für dich bedeutet das nun das Abarbeiten folgender Schritte:

- Wechsle in den Vorlagenordner (also das passende Fenster)
- Checke den richtigen Stand aus: `git checkout Step_2`
- Kopiere die Datei: `cp story1.md ../buch/`
- Wechsle in den Buch-Ordner (also das andere Fenster)
- Checke den Status: `git status`

```
# Ausgabe
Auf Branch main

Noch keine Commits

Zum Commit vorgemerkte Änderungen:
  (benutzen Sie "git rm --cached <Datei>..."
   zum Entfernen aus der Staging-Area)
    neue Datei:     story1.md                                  ①

Änderungen, die nicht zum Commit vorgemerkt sind:
  (benutzen Sie "git add <Datei>...",
   um die Änderungen zum Commit vorzumerken)
  (benutzen Sie "git restore <Datei>...", um die Änderungen im
   Arbeitsverzeichnis zu verwerfen)
    geändert:       story1.md                                  ②
```

① Die alte Version der Datei ist zum Commit vorgemerkt
② Die neuere Version ist noch nicht im Stage!

Wieder werden mögliche Befehle angegeben, die in diesem Zustand sinnvoll sein können!

Hinweis
Wenn du versuchst, die Datei mit `git rm --cached story1.md` aus dem Stage zu löschen, bekommst du eine Fehlermeldung – es könnte nämlich sein, dass der Vorgang zu Datenverlust führt!

```
error: die folgende Datei hat zum Commit vorgemerkte Änderungen
      unterschiedlich zu der Datei und HEAD:
         story1.md
(benutze -f, um die Löschung zu erzwingen)
```

Bei `git restore story1.md` gibt es keine solche Warnung! Die Datei verbleibt dabei auch im Stage unverändert erhalten, aber die Arbeitsversion wird überschrieben.

Nun kannst du erstmalig eine Differenz zwischen den Versionen der Datei erkennen:

```
git diff story1.md
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

6.2.0.1 Analyse der Differenz

```
diff --git a/story1.md b/story1.md                          ①
index 9c818e7..3126a18 100644                               ②
--- a/story1.md                                             ③
+++ b/story1.md
@@ -25,3 +25,63 @@                                          ④
entdeckte sie den geheimnisvollen Tunnel.                   ⑤
  der gerade von einem seiner Flüge zurückgekehrt
  war und sich auf dem Ast des Apfelbaums niederließ.       ⑥

  +„Ich glaube, es ist ein Tunnel!" miaute Luna             ⑦
  +aufgeregt. „Vielleicht führt er zu einem verborgenen
```

① Welche Dateien werden verglichen? `a/` und `b/` sind nur Kennzeichner der Dateien. `a/` ist die ältere (z.B. Stage) und `b/` die neuere Datei (z.B. *Working-Directory*).
② Bei 9c818e7 und 3126a18 handelt es sich um die Hash-IDs der beiden Dateien, die verglichen werden (dazu unten mehr).
③ Soll erneut klarstellen `---` ist alt, `+++` ist neu.
④ Die Zeile heißt *Hunk Header*. Er beginnt und endet mit `@@`
⑤ Die Ausgabe beginnt in beiden Dateien in Zeile 25 (Hunk-Header). und stellt 3 Zeilen der alten und 63 Zeilen der neuen Datei dar.
Bei `diff` werden normalerweise 3 Zeilen vor ud nach der Änderung mit angegeben.
⑥ Bis hier keine Änderung.
⑦ Diese Zeilen stehen nur in der neuen Datei – wegen des „+“ Zeichens.

Leider ist das Format vom *Hunk Header* etwas kryptisch! Er gibt immer an, in welcher Zeile der Vergleich beginnt und wie viele Zeilen betroffen sind. Bei `@@ -3,12 +3,14 @@` beginnt der relevante Bereich ab Zeile 3 und in der ursprünglichen Version lagen 12 Zeilen vor, in der neuen Version sind es jetzt 14. Die Änderungen im Detail folgen dann in den nächsten Zeilen.

Nun wird die Datei wieder zum Stage hinzugefügt und dann committet:

Eingabe

```
git add story1.md
git commit -m 'Guter Ansatz'

# Ausgabe
[main (Root-Commit) 5229c6b] Guter Ansatz
 1 file changed, 4 insertions(+)
 create mode 100644 story1.md
```

Dieser Arbeitszyklus von *add, status, diff, commit* stellt den Kern der Arbeit mit *git* dar, zumindest wenn du alleine arbeitest. Allerdings gibt es noch weitere, sehr wichtige Techniken (push, fetch, pull), die später bei der Arbeit im Team eine große Rolle spielen werden.

6.3 Beginn von Story 2

Du hast nun die Datei `story1.md` so weit fertig gestellt und möchtest an einer anderen Kurzgeschichte arbeiten. Du beschließt, dass fertige Geschichten in Branch `main` liegen sollen und das aktuelle Buch aber während der Entwicklung einen eigenen Branch bekommt.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

In der Softwareentwicklung sprechen wir hier von *fertigen Produkten*, die an den Kunden ausgeliefert werden und von *Experimenten, Features, Bugfixes, …*, die parallel dazu entwickelt werden müssen. Durch ein Experiment darf die aktuell lauffähige Version nicht beeinträchtigt werden! Du brauchst also einen *Entwicklungszweig* und einen *Auslieferungszweig*.

```
# Zweig erstellen
git switch -c entwicklung_story_2
```

Git teilt dir mit, dass du dich nun in diesem *Zweig* (=Branch) befindest. Du siehst hier immer noch deine aktuelle *Story 1*! Sie ist ebenfalls in dem neu erstellten Branch enthalten, du hast aber nicht vor, an ihr zu arbeiten. Das wäre zwar möglich, würde aber mein Szenario hier stören!

Nun beginnst du mit *Story 2*, indem du sie aus dem Vorlagenordner kopierst (weil wir nicht schreiben).

- Wechsle in den Vorlagenordner (Fenster)
- Checke den richtigen Stand aus: `git checkout Step_3`
- Kopiere die Datei: `cp story2.md ../buch/`
- Wechsle in den Buch-Ordner (Fenster)
- Checke den Status: `git status`

Das ist dein Einstieg in *Story 2*. Für den Überblick solltest du einen Blick in den aktuellen Text werfen – zum Beispiel mit `nano` oder mit `less story2.md` (mit `q` kommst du wieder heraus!)

Du bist mit deinem bisherigen Werk ganz zufrieden, hast aber zwei Ideen, wie du weiter machen könntest. Aus diesem Grund erstellst du zunächst einen Commit

```
git add story2.md
git commit -m "Story 2 begonnen"
```

und legst für jede Idee einen Branch an:

```
git switch -c story2_idee_1
git switch entwicklung_story_2
git switch -c story2_idee_2
```

Nun nicht den Überblick verlieren! Im Branch `story2_idee_1` arbeitest du an deiner ersten Idee von `story2.md` weiter, im Branch `story2_idee_2` probierst du einen alternativen Verlauf.

Da wir nur kopieren und nicht schreiben, heißt das für dich:

- Im Buch-Ordner: `git switch story2_idee_1`
- Wechsle in den Vorlagenordner
- Checke den richtigen Stand aus: `git checkout Step_4`
- Kopiere die Datei: `cp story2.md ../buch/`
- Wechsle in den Buch-Ordner
- `git add story2.md`
- `git commit -m "Inhalt nach Idee1 fortgeführt"`

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Da du aber noch nicht sicher bist, ob diese Version verwenden willst, arbeitest auch Idee 2 aus.

- Im Buch-Ordner: `git switch story2_idee_2`
- Wechsle in den Vorlagenordner
- Checke den richtigen Stand aus: `git checkout Step_5`
- Kopiere die Datei: `cp story2.md ../buch/`
- Wechsle in den Buch-Ordner
- `git add story2.md`
- `git commit -m "Inhalt nach Idee 2 fortgeführt"`

Nun hast du das *Problem*, dass drei Versionen von `story2.md` existieren:

1. Version 1 im Entwicklungszweig.
   Sie ist die Stufe vor der Neubearbeitung.
2. Version 2 im Branch für Idee 1
3. Version 3 im Branch für Idee 2

Du bist unentschieden und siehst dir die 3 Versionen noch einmal in Ruhe an.

```
git switch entwicklung_story_2  # in Ruhe durchlesen - die Basis
git switch story2_idee_1        # in Ruhe durchlesen - Idee 1
git switch story2_idee_2        # in Ruhe durchlesen - Idee 2
```

Dabei fällt dir ein *grober Rechtschreibfehler* im Entwicklungszweig (Basisiversion) auf, den du sofort korrigierst. Bei uns ist es das Wort *weiße*, das sich anstelle von *weise* in den Text geschummelt hat.

- Kontrolliere, dass du im Branch `entwicklung_story_2` bist
  (`gti branch` oder `git status`)
- Wechsle in den Vorlagenordner
- `git checkout Step_6`
- `cp story2.md ../buch/`

Beachte, dass diese Änderung in den anderen beiden Branches `story2_idee_1` und `story2_idee_2` *nicht* ankommt!

Deine Änderungen an der Datei kontrollierst du kurz mit `diff`:
(Das ist ein Vergleich von *Working-Directory* mit dem *Stage*.)

```
git diff

# Ausgabe
diff --git a/story2.md b/story2.md
index 1e193c3..bb2c446 100644
--- a/story2.md
+++ b/story2.md
@@ -9,7 +9,7 @@ bereits am Horizont. Freddy liebte solche Abende,
 denn sie versprachen Abenteuer und Entdeckungen.
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
 Freddy war nicht allein. Seine beste Freundin,
-Shelly, die weiße Schildkröte, lebte ebenfalls am
+Shelly, die weise Schildkröte, lebte ebenfalls am
 Teich. Shelly war für ihre ruhige und besonnene
 Art bekannt, aber sie konnte auch sehr mutig sein,
 wenn es darauf ankam.
```

(Probiere auch `diff --color-words` aus!)

Mit den folgenden Befehlen übernimmst du nun die Änderung.

```
git add .
git commit -m "weiße - weise"
```

Nun zeigt dir der Befehl

```
git diff --color-words entwicklung_story_2 story2_idee_1
```

nichts mehr an, denn *Working-Directory* und *Stage* sind identisch. dass ein Austausch der Wörter vorgenommen wird (kenntlich gemacht):

Nun möchtest du die beiden Branches zusammenführen.

```
git merge story2_idee_1  # erster Merge
```

Beachte, dass jetzt ein Fast-Forward-Merge nicht möglich ist, da sich beide Branches von ihrem gemeinsamen Vorfahren weg entwickelt haben! Es wird also einen *Merge-Commit* geben, der beide Branches zusammenführt.

git erwartet von dir eine Begründung für den Merge. Gib also *Vorläufig ok* ein, speichere und schließe den Editor.

Du bist erfreut, dass dieser Merge problemlos funktioniert.

Allerdings bist du am Überlegen, ob das mit der Schildkröte so eine gute Idee ist. Du entscheidest dich, eine *Riesen*schildkröte daraus zu machen. Aus Sicherheitsgründen willst du das aber zuerst im Ideen-Zweig ändern!

Routinemäßig machst du vorher einen *diff*.

```
git diff --color-words entwicklung_story_2 story2_idee_1
```

Beachte, dass du aktuell die Dateiversionen vergleichst, die in den Commits stehen!

Die Idee wurde beim Merge übernommen, der Rechtschreibfehler ist der einzige Unterscheid zwischen den Versionen. Wenn du also in den Branch `story2_idee_1` wechselst, dann ist der Fehler immer noch da!

Du wechselst also in den Ideen-Branch und führst die Änderung mit der Riesenschildkröte durch.

Verwende *suchen / ersetzen* oder im Terminal:

```
git switch story2_idee_1
sed -i -e "s/Schildkröte/Riesenschildkröte/g" story2.md
```

(Alternativ kannst du auch die letzte Version aus der Vorlage kopieren.)

Zur Kontrolle machst du den Diff erneut:

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
# Vergleich der Banches - also das was in den letzen Commits steht
git diff --color-words entwicklung_story_2 story2_idee_1
```

Seltsam … du siehst die *Riesen*schildkröte nicht im Diff!
Aber eigentlich ist das klar, denn es werden ja die Versionen *in den Commits* verglichen und diese neue Version ist noch nicht im Commit. Momentan haben wir also die Situation:

```
# im Branch story2_idee_1
Work:   Riesenschildkröte
Stage:  LEER
Commit: Schildkröte
```

Ein einfaches

```
git diff
```

vergleicht die *aktuelle* Version im *Working-Directory* gegen den Commit, weil der Stage aktuell leer ist (siehe folgender Abschnitt)! Hier siehst du die Riesenschildkröte und die einfache Schildkröte.

Wenn dich diese ganzen *diff*s verwirren … das ist normal. Du kannst auch kurz zum Ende dieses Abschnitts springen. Dort erkläre ich das noch einmal. Verliere aber nicht den Faden hier im Buch!

6.3.1 Weiter im Buch

Die Version mit der Riesenschildkröte willst du nun wieder in den Entwicklungsstand mergen. Dafür musst du vorher wieder einen Commit machen.

```
git add story2.md
git commit -m "Riesenschildkröte statt Schildkröte"
```

Du machst einen erneuten Vergleich der Branches mit Diff

```
# Wieder Branch-Vergleich
git diff --color-words entwicklung_story_2 story2_idee_1
```

Da du den Schreibfehler ja nur im Entwicklungszweig verbessert hast, siehst du folgende Situation:

Die *weiße Riesenschildkröte* (Schreibfehler aber richtiges Tier) im Ideen-Branch und *weise Schildkröte* (Richtige Schreibung aber falsches Tier) im Entwicklungs-Branch.
In dieser Situation ist *git* überfordert. Jeder der beiden Texte ist falsch und deshalb musst du als Benutzer entscheiden, was am Ende in der Datei stehen soll – aber das sagt dir *git* beim Merge!

```
git switch entwicklung_story_2
git merge story2_idee_1
```

Nun siehst du eine fette Fehlermeldung, weil die Versionen nicht mehr zusammenpassen! Gerade für Anfänger ist das eine unheimliche Situation, in der man anscheinend viel kaputt machen kann … kann man ja auch. Es ist aber halb so tragisch!

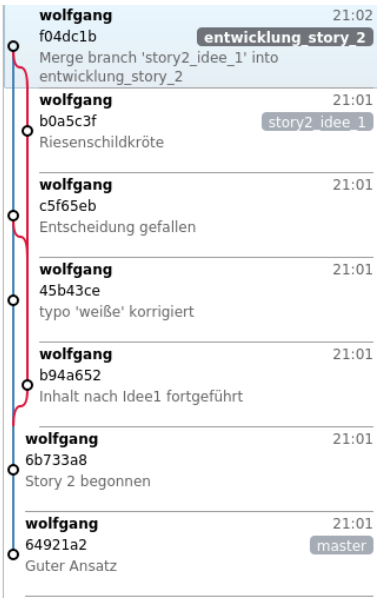
© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Öffne die entsprechende Datei im Editor. Du erkennst sofort die Stelle, an der Änderungen erfolgt sind:

```
Freddy war nicht allein. Seine beste Freundin,
<<<<<<< HEAD
Shelly, die weise Schildkröte, lebte ebenfalls am
=======
Shelly, die weiße Riesenschildkröte, lebte ebenfalls am
>>>>>>> story2_idee_1
Teich. Shelly war für ihre ruhige und besonnene
```

Lösche einfach die Markierungen und korrigiere den Text passend.
Wieder ein *add* und ein *commit* und das Problem ist vom Tisch!

Das nachfolgende Diagramm zeigt den Ablauf der Operationen zwischen den beiden Zweigen (Idee 2 wurde ausgeblendet, weil du mit ihr bisher nichts mehr gemacht hast).



Gut – das war ein langer Weg mit vielen Abzweigungen. Es ist normal, diesen Abschnitt mehrfach lesen zu müssen!

6.3.2 Fazit

Branches erlauben gefahrloses Arbeiten an Ideen. Baut man die Szenarien nicht zu komplex auf, dann gibt auch meist keine Probleme beim Merge.

Immer wenn du Änderungen in einem Zweig vornimmst, die du auch im anderen Branch benötigst (weiße/weise) kannst du den Merge auch in die andere Richtung ausführen. Das ist ein übliches Verfahren, weil man als Entwickler immer auf dem aktuellen Stand des Auslieferungszweigs sein muss, auch wenn du weiter an deinem Experiment arbeitest.

Die Vergleiche mit *diff* sind definitiv gewöhnungsbedürftig. Machst du häufige Commits auch schon bei kleinen Änderungen, so bekommst du auch übersichtliche *diffs*. Wenn du nicht alle Dateien vergleichen willst, kannst du das auch einschränken:

```
git diff <commit>..<commit> <datei>
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Der Nachteil von vielen Commits sind allerdings … viele Commits. Es gibt aber Möglichkeiten nachträglich Commits zusammenzufassen um die Entwicklungsgeschichte zu verschlanken. Diesen Vorgang nennt man *squashen* und er ist eine Variante von *rebase*.

6.3.3 Genauerer Blick auf Diff

Diese Diffs sind verwirrend und deshalb betrachten wir noch ein paar einfachere Beispiele.

Variante 1:
Im Commit, dem *Stage* und dem *Working-Directory* befinden sich (unterschiedliche) Versionen einer Datei:

```
cd
rm -rf labor
git init labor
cd labor
git branch -m main

echo "Ich bin im commit" > datei.txt
git add datei.txt
git commit -m "Version 1"

echo "Ich bin im stage" > datei.txt
git add datei.txt

echo "Ich bin im work" > datei.txt
```

Nun wollen wir die einzelnen Versionen miteinander vergleichen. Das sind 3 Befehle:

Working-Directory mit *Stage*

```
git diff datei.txt

# Ausgabe
diff --git a/datei.txt b/datei.txt
index 4a81434..dc87399 100644
--- a/datei.txt
+++ b/datei.txt
@@ -1 +1 @@
-Ich bin im stage
+Ich bin im work
```

Stage mit *commit*

```
git diff --cached datei.txt
# oder git diff --staged datei.txt

# Ausgabe
diff --git a/datei.txt b/datei.txt
index 07d0c2d..4a81434 100644
--- a/datei.txt
+++ b/datei.txt
@@ -1 +1 @@
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
-Ich bin im commit
+Ich bin im stage
```

Working-Directory mit *commit* (Hash bekannt, oder HEAD)

```
git diff HEAD datei.txt

# Ausgabe
diff --git a/datei.txt b/datei.txt
index 07d0c2d..dc87399 100644
--- a/datei.txt
+++ b/datei.txt
@@ -1 +1 @@
-Ich bin im commit
+Ich bin im work
```

Variante 2:
Im Commit und dem *Working-Directory* befinden sich (unterschiedliche) Versionen einer Datei. *Der Stage ist aktuell leer.*

```
cd
rm -rf
git init labor
cd labor
git branch -m main

echo "Ich bin im commit" > datei.txt
git add datei.txt
git commit -m "Version 1"

# KEIN STAGE

echo "Ich bin im work" > datei.txt
```

Working-Directory mit *commit*

```
git diff datei.txt

# Ausgabe
# Weil der Stage leer ist, wird gegen den
# Commit vergleichen
diff --git a/datei.txt b/datei.txt
index a0a62ed..dc87399 100644
--- a/datei.txt
+++ b/datei.txt
@@ -1 +1 @@
-Ich bin im  commit
+Ich bin im work
```

Stage mit *commit*

```
git diff --cached datei.txt
# oder git diff --staged datei.txt
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
# Ausgabe
# LEER - Nichts im Stage!
```

Working-Directory mit *commit* (Hash bekannt, oder HEAD)

```
git diff HEAD datei.txt

# Ausgabe (wie oben )
diff --git a/datei.txt b/datei.txt
index a0a62ed..dc87399 100644
--- a/datei.txt
+++ b/datei.txt
@@ -1 +1 @@
-Ich bin im commit
+Ich bin im work
```

Hier sind die Fälle 1) und 3) also identisch.
Wie kann man also sicher sein, welche Dateien ein reines `git diff` vergleicht?
Liefert `git status` einen leeren *Stage*, so wird mit dem Commit verglichen. Ist eine Datei im *Stage*, so wird gegen diese Datei verglichen.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

7 Fortgeschritten: Merge, Rebase, Squash

Im letzten Abschnitt hast du einfaches Arbeiten mit Branches gelernt und wie man sie mit Hilfe eines *Merge* zusammengeführt. Das wollen wir uns jetzt noch genauer ansehen. Offen geblieben sind folgende Fragen:

- Details zu Merges
- Was passiert mit alten Branches?
- Wie sieht die Alternative zu *Merge* aus?
- Was mache ich mit zu vielen Commits?

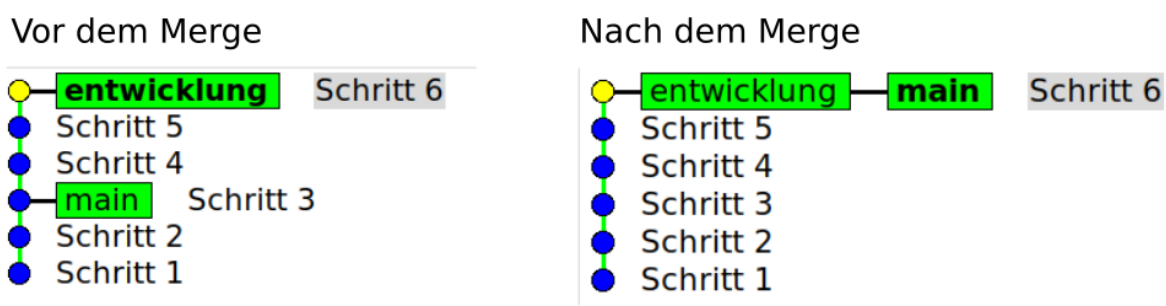
7.1 Verschiedene Merges

In meinem Alltag treten meist drei Fälle auf, da ich in der Regel alleine entwickle:

- Während ich am Entwicklungs-Branch arbeite, passiert auf dem Auslieferungsbranch (hier *main*) nichts, außer vereinzelte Bug-Fixes.
- Ich entwickle an einem völlig anderen Feature, das den *main*-Branch aktuell gar nicht betrifft.

7.1.1 Fast-Forward

Befindest du dich auf dem (unveränderten) *main*-Branch und holst den Entwicklungsbranch mit Merge, so wird der Pointer *main* auf den letzten Commit des Entwicklungs-Branch geschoben. In dieser Situation wird der Branch im Diagramm nicht einmal als visuelle Abzweigung dargestellt:

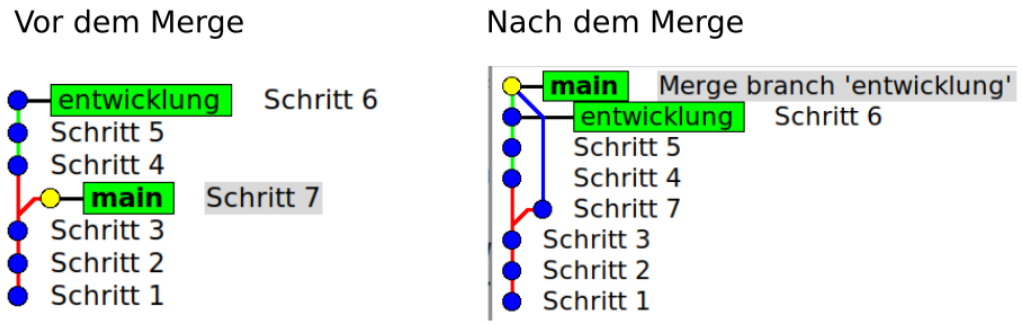


Erkennbar sind die grünen Rechtecke, die die Enden der Branches kennzeichnen. Im Bild links sind die Branches getrennt, im Bild rechts sind beide Branch-Enden nach dem Merge an der gleichen Stelle. Wie du siehst, hat sich beim Merge die Anzahl Commits nicht verändert.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

7.1.2 Drei-Wege Merge

Wenn sich der *main*-Branch parallel zum Entwicklungsbranch weiterentwickelt, sieht die Sitation ganz anders aus!



Du siehst, dass sich der *main*-Branch noch *Schritt 3* zu *Schritt 7* weiterentwickelt hat (neuer Commit). Denk immer daran, dir die Branchnamen wie Etiketten am jeweils letzten Commit des Branchs vorzustellen!
Beim Merge entsteht ein neuer Commit, der die Spitze beider Branches darstellt. Du erkennst auch, dass im Diagramm die grünen Spitzen der Branches nicht am gleichen Commit sitzen!

Durch `git log --decorate --oneline --graph` (merken als „git log dog“) wird diese Sitation auch im Terminal passend dargestellt:

```
*   70c1b21 (HEAD -> main) Merge branch 'entwicklung' into main
|\
| * 4a6e3a5 (entwicklung) Schritt 6
| * edaeaff Schritt 5
| * 508019b Schritt 4
* | 2ff4a45 Schritt 7
|/
* 29d2bd4 Schritt 3
* 964ee6f Schritt 2
* 66dcd4f Schritt 1
```

(Welcher Branch (main oder entwicklung) als Abzweigung erscheint, entscheidet die jeweilige Git-Software abhängig von der aktuellen Situation!)

7.2 Alte Branches

Die Commits eines Branches stellen eine Dokumentation des Projektverlaufs dar, die für das Verständnis der erfolgten Änderungen eventuell notwendig ist. Zu viele Branches und Commits erschweren auf der anderen Seite den Überblick.

Es geht also darum, alte Branches zu löschen, die *wichtigen* Commits aber zu behalten. Für den schulischen Kontext ist das aber weniger relevant.

Um einen lokalen Branch zu löschen, gibt es folgenden Befehl:

```
git branch -d <name>
```

Er scheitert aber, wenn *git* den Branch noch nicht als abgeschlossen betrachtet. Das ist der Fall, wenn die Ergebnisse des Branchs noch nicht durch einen Merge *in Sicherheit* gebracht wurden. Hier hilft `-D` an Stelle von `-d`.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Probieren wir das beim letzten Szenario aus. Sieh dir die Abbildung oben noch einmal an und dann lösche den Branch

```
git branch -d entwicklung
```

Du siehst eine minimale Änderung

```
*   70c1b21 (HEAD -> main) Merge branch 'entwicklung' into main
|\
| * 4a6e3a5 Schritt 6   <<< hier fehlt (entwicklung)
| * edaeaff Schritt 5
| * 508019b Schritt 4
* | 2ff4a45 Schritt 7
|/
* 29d2bd4 Schritt 3
* 964ee6f Schritt 2
* 66dcd4f Schritt 1
```

Das *Etikett* vom Entwicklungsbranch wurde gelöscht, die Commits bleiben aber in exakt der gleichen Anordnung erhalten! Der Branch ist nicht mehr über seinen Namen zugänglich, die History ist aber weiterhin vorhanden, d.h. ein *checkout* der Commits ist immer noch möglich.

Anders ist das allerdings, wenn du einen Branch löschst, der *nicht* in einen Merge verwickelt ist. Er existiert dann zwar anscheinend weiter, wird aber nach einiger Zeit vom Garbage-Collector (GC) von *git* endgültig entfernt. (Der Zeitrahmen ist einstellbar). Im Prinzip passiert das mit allen Commits, die nicht nicht mehr *erreichbar* sind.

Beispiel

Das Script `material/git_core/07_merge/branch_delete.sh` erzeugt dir diese Situation:

```
A - B - C - D  <-- main
      \ E - F - G  <-- feature
```

Willst du den Branch *feature* löschen, bekommst du eine Warnung.

```
git branch -d feature

# Ausgabe
error: the branch 'feature' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature'
```

Löschst du ihn dann dennoch `git branch -D feature`, so wird der Pointer auf G entfernt:

```
A - B - C - D  <-- main
      \ E - F - G
```

Da jeder Commit nur seinen Vorgänger kennt, kommst du zwar von D nach A, aber nicht *rückwärts* nach E, F oder G.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

7.2.1 „Fun“-Fact

Wenn du nun meinst, du hättest das verstanden … warum sollte es dir besser gehen als mir an dieser Stelle? Spielen wir das Szenario erneut durch:

- Script laufen lassen `material/git_core/07_merge/branch_delete.sh`
- Branches anzeigen `git log --oneline --graph --all --decorate`
- Feature-Branch löschen `branch -D feature`
- Branch anzeigen `git log --oneline --graph --all --decorate`

… der Branch ist noch sichtbar? WARUM?

Das Flag “`--all`” sorgt dafür, dass alle Commits angezeigt werden, auf die es noch in irgendeiner Weise Referenzen im System gibt. Ich hatte an dieser Stelle gedacht, sie seien alle weg. Sicherheitshalber löste ich noch eine manuelle Bereinigung des Systems aus (war bei uns nicht Thema):

```
git reflog expire --expire=now --all
git gc --prune=now
```

Aber der Branch war immer noch sichtbar … Was nun?
Die Lösung war, dass wirklich noch Referenzen auf die Commits exisierten! Schuld war ich selbst, weil ich es besonders gut gemeint hatte und den Commits jeweils *Tags* zugeordnet hatte. In einem früheren Kapitel hatte ich explizit *Tags* als einen der 4 Objekttypen von *git* angegeben und diese Tags sind auch wirklich Referenzen auf die Commits. Nach der Löschung der Tags:

```
git tag -d E F G
```

war auch wirklich der Branch verschwunden! Willst du sicher sein, dass auch wirklich **alle** Altlasten entfernt wurden, kannst du erneut eine manuelle Bereinigung des Systems ausführen. Das würde ich aber nicht empfehlt, weil es von *git* egentlich so nicht gedacht ist.

7.3 Rebase

Neben dem *Merge* trifft man auch oft auf den *Rebase* um Branches zusammenzuführen. Er funktioniert allerdings deutlich anders und ist wegen seiner vielfältigen Möglichkeiten (z.B. Reihenfolge der Commits ändern, …) ein Werkzeug für fortgeschrittene Benutzer. Mit seiner Hilfe können auch mehrere Commits zusammengefasst werden, um die Branches zu verkürzen. Gerade bei der Zusammenarbeit im Team kann das aber sehr problematisch werden, wenn man nicht ganz genau weiß, was man macht. Das liegt daran, dass sich die Hashwerte von Commits ändern können und wenn sich ein anderer Mitarbeiter den früheren Stand auf den eigenen Rechner geholt hat, dann passt nichts mehr zusammen.

https://www.youtube.com/watch?v=CtyLg10aHN0
https://www.youtube.com/watch?v=1TNK-OkaeII

Bei einem Rebase wird der entsprechende Branch *verpflanzt* – d.h. gewissermaßen *ausgerupft* und an anderer Stelle wieder *angedockt*.
Bei diesem Vorgang werden die Hashwerte aller Commits im Branch geändert.

Betrachte das nachfolgendes Script (`material/git_core/08_rebase/rebase.sh`). Nach dem ersten Commit wird ein Branch *arbeit* mit mehreren Commits erstellt und im Anschluss ein Commit wieder im *main*-Branch, **der eine andere Datei betrifft**.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
#!/bin/bash
cd
rm -rf rebase_lab
git init rebase_lab
cd rebase_lab
git branch -m main

echo "start" >> datei.txt
git add . && git commit -m "Start"

# Branch-Wechsel
git switch -c arbeit

for i in {1..10};do
  echo "inhalt $i" >> datei.txt
  git add . && git commit -m "Inhalt $i"
done

# Branch-Wechsel
git switch main

echo "Zwischenstopp" >> datei2.txt
git add . && git commit -m "Zwischenstopp"
```

```
cd /tmp
rm -rf rebase_lab
git init rebase_lab
cd rebase_lab
git branch -m main

echo "start" >> datei.txt
git add . && git commit -m "Start"

# Branch-Wechsel
git switch -c arbeit

for i in {1..10};do
  echo "inhalt $i" >> datei.txt
  git add . && git commit -m "Inhalt $i"
done

# Branch-Wechsel
git switch main

echo "Zwischenstopp" >> datei2.txt
git add . && git commit -m "Zwischenstopp"
```

Ziel ist nun ein *rebase* von *arbeit* auf *main*.

Vor dem Rebase

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
* 0834b9d (arbeit) Inhalt 10
* 4cdd087 Inhalt 9
* ebfa1db Inhalt 8
* d078d5a Inhalt 7
* a65af00 Inhalt 6
* cfb81ab Inhalt 5
* 7fc30db Inhalt 4
* 493cb05 Inhalt 3
* 4648598 Inhalt 2
* cff48d7 Inhalt 1
| * 1af4a22 (HEAD -> main) Zwischenstopp
|/
* af7e60c Start
```

Nun der Rebase

```
# aktueller Branch ist arbeit
git switch arbeit
git rebase main
git log --oneline --decorate --graph --all

# Ausgabe
* 43add76 (HEAD -> arbeit) Inhalt 10
* 117f222 Inhalt 9
* 9057374 Inhalt 8
* 71ec034 Inhalt 7
* 66852df Inhalt 6
* f144368 Inhalt 5
* cdba6cf Inhalt 4
* d059b1b Inhalt 3
* d1b3718 Inhalt 2
* c591dc2 Inhalt 1   << NEUE HASHES
* 1af4a22 (main) Zwischenstopp << Hash unverändert
* af7e60c Start                << Hash unverändert
```

Nach dem *rebase* befindet du dich immer noch auf dem Branch *arbeit* (Siehe HEAD Pointer), die *Abzweigung* nach Start ist verschwunden. Der Branch `arbeit` hängt mit seinen Commits und neuen Hashwerten also direkt am Commit mit dem Hash 1af4a22 aus dem Branch `main`.

Damit ist der Rebase aber noch nicht abgeschlossen! Du musst nun in den *main*-Branch wechseln und einen Merge ausführen

```
git switch main
git merge arbeit

# Ausgabe

git merge arbeit
Aktualisiere 1af4a22..43add76
Fast-forward
 datei.txt | 10 ++++++++++
 1 file changed, 10 insertions(+)
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Damit ist der HEAD wieder an der korrekten Stelle:

```
git log --oneline --decorate --graph --all

# Ausgabe
* 43add76 (HEAD -> main, arbeit) Inhalt 10
* 117f222 Inhalt 9
* 9057374 Inhalt 8
* 71ec034 Inhalt 7
* 66852df Inhalt 6
* f144368 Inhalt 5
* cdba6cf Inhalt 4
* d059b1b Inhalt 3
* d1b3718 Inhalt 2
* c591dc2 Inhalt 1
* 1af4a22 Zwischenstopp
* af7e60c Start
```

Wie du siehst, ist der Branch *arbeit* aber noch existent! Lösche ihn mit folgendem Befehl:

```
git branch -d arbeit
git log --oneline --decorate --graph --all

# Ausgabe
* 43add76 (HEAD -> main) Inhalt 10
* 117f222 Inhalt 9
* 9057374 Inhalt 8
* 71ec034 Inhalt 7
* 66852df Inhalt 6
* f144368 Inhalt 5
* cdba6cf Inhalt 4
* d059b1b Inhalt 3
* d1b3718 Inhalt 2
* c591dc2 Inhalt 1
* 1af4a22 Zwischenstopp
* af7e60c Start
```

Damit ist der Rebase komplett beendet und das Repository wieder aufgeräumt. Das lief so reibungslos, weil der Commit *Zwischenstop* eine andere Datei betroffen hat (`datei2.txt`). Hätten Änderungen an `datei.txt` stattgefunden, so hätten wir einen Konflikt bekommen, den wir hätten lösen müssen.

Hinweis
Oft wird gefragt, warum man einen *Rebase* machen sollte, wenn danach immer noch ein *Merge* nötig ist. Im Prinzip wird das der Graph des Repository *schlanker* und damit übersichtlicher. Der *Merge* ist auch nicht unbedingt nötig, da es ja ein ff-Merge ist und damit ginge auch ein einfaches `git switch main && git reset --hard HEAD`.

Was machen ich bei einem Konflikt?

Führe das Script `rebase_mit_konflikt.sh` aus

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
#!/bin/bash
cd
rm -rf rebase_lab
git init rebase_lab
cd rebase_lab
git branch -m main

echo "start" >> datei.txt
git add . && git commit -m "Start"

# Branch-Wechsel
git switch -c arbeit

for i in {1..10};do
  echo "inhalt $i" >> datei.txt
  git add . && git commit -m "Inhalt $i"
done

# Branch-Wechsel
git switch main

echo "Zwischenstopp" >> datei.txt     # <<<< HIER Konflikt
git add . && git commit -m "Zwischenstopp"
```

Versuche nun einen Rebase:

```
git rebase main

# Ausgabe
automatischer Merge von datei.txt
KONFLIKT (Inhalt): Merge-Konflikt in datei.txt
error: Konnte 2f5968f... (Inhalt 1) nicht anwenden
Hinweis: Resolve all conflicts manually, mark them as resolved with
Hinweis: "git add/rm <conflicted_files>", then run "git rebase --continue".
Hinweis: run "git rebase --continue".
Hinweis: You can instead skip this commit: run "git rebase --skip".
Hinweis: To abort and get back to the state before "git rebase",
Hinweis: run "git rebase --abort".
Konnte 2f5968f... (Inhalt 1) nicht anwenden
```

Die Hinweiszeilen beschreiben eigentlich sehr genau, was zu tun ist:

- Konflikt manuell lösen
- `git add datei.txt`
- `git rebase --continue` ausführen
- Commit-Nachricht anpassen

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

7.3.1 Merge oder Rebase

Bei *merge* kann die History verzweigter aussehen, bei *rebase* können sich Commit-Hashes ändern.

Als Faustregel gilt, dass du die Finger vom Rebase lassen solltest, wenn irgendwer außer dir Zugriff auf den betroffen Branch hat (Team-Arbeit). Solange du nur lokal auf deinem Rechner arbeitest gibt es (außer Konflikten) kein Problem.

7.4 Squash

Squash ist kein eigenständiger Befehl, sondern ein Parameter von *rebase*. Angenommen, du hast eine ganze Liste von filigranen Commits erzeugt, die du im Nachhinein zu einem einzigen Commit zusammenfassen willst. In diesem Fall führst du einen *interaktiven Squash* durch, möglich wären:

```
git rebase -i HEAD~3 # 3 Commits, rückwärts von HEAD
git rebase -i <hash> # der älteste Commit zum Squashen
...
```

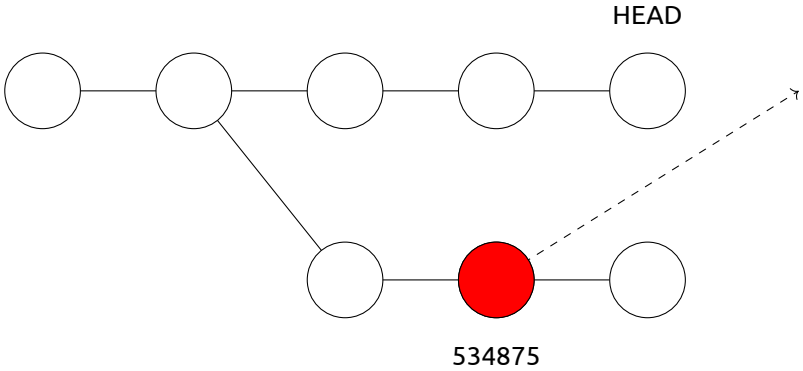
Als Reaktion wird dein Editor gestartet, der dir in den ersten Zeile die zum Squash verfügbaren Commits anzeigt. Angenommen, es geht um 3 Commits, dann ersetzt du bei allen **bis auf den neuesten** Commits das *pick* durch ein *s* (für Squash).
Mit dem Speichern der Datei wird dies durchgeführt und du bekommst ein neues Editor-Fenster für eine Commit-Message. Hast du diese gespeichert, ist der Squash beendet.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

8 Cherry Picking

8.1 Einfach und problemlos

Dieser Befehl klingt nach hoher Expertise, ist aber im Vergleich zu anderen Techniken relativ leicht zu verstehen. Kurz gesagt ermöglicht es Cherry-Pick, sich Commits aus anderen Branches in den eigenen Branch zu kopieren, d.h. die vorgenommenen Änderungen auf den eigenen anzuwenden. Das ist Hilfreich, wenn zum Beispiel in einem Branch ein Bug behoben wird, der auch im eigenen Branch benötigt wird.



Der zugehörige Befehl würde – ausgehend vom aktuellen Branch (HEAD) lauten:

```
git cherry-pick 534875
```

Als Ergebnis entsteht dadurch am Ende ein komplett neuer Commit mit neuem Hash im aktuellen Branch.

Das Script `material/09_cherry/cherry_pick.sh` liefert genau dieses problemlose Szenario. *Problemlos* deshalb, weil in beiden Branches komplett unterschiedliche Dateien bearbeitet werden. Damit können die Änderungen einfach übernommen werden:

```
#!/bin/bash

# Zwei Branches, unterschiedliche Dateien

# Putzen
cd
rm -rf cherry_labor

# Lokales Repo erstellen
mkdir cherry_labor
cd cherry_labor
git init
git branch -m main

#############
# Auf branch main
echo "<h1>Intro</h1>" > index.html
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
git add index.html
git commit -m "initial"

echo "<h2>Einleitung</h2>" >> index.html
git add index.html
git commit -m "Einleitung"

#############
# wechsel zu development
git switch -c development

echo "<div class='nav'> ... </div> " > nav.html
git add nav.html
git commit -m "Nav Start"

echo "<script>...</script> " >> nav.html
git add nav.html
git commit -m "javascript begonnen"

#############
# zurück zu main
git switch main

echo "<h2>Hauptteil</h2>" >> index.html
git add index.html
git commit -m "Hauptteil"

git log --oneline --decorate --graph --all
```

Ergebnis

```
* 0c10748 (development) javascript begonnen
* e103d4b Nav Start     <<<<<<<<< Das wollen wir
| * 97bf335 (HEAD -> main) Hauptteil  <<<<<<<< an dieser Stelle
|/
* b2228f5 Einleitung
* 57085c3 initial
```

Der benötigte Hash lautet e103d4b und damit geht ganz einfach

```
git cherry-pick e103d4b

# Ausgabe
[main 2e5c343] Nav Start
 Date: Wed Jan 7 22:51:49 2026 +0100
 1 file changed, 1 insertion(+)
 create mode 100644 nav.html
```

Und zur Kontrolle

```
git log --oneline --decorate --graph --all

# Ausgabe
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
* 2e5c343 (HEAD -> main) Nav Start    <<<<<< neuer Hash
* 97bf335 Hauptteil
| * 0c10748 (development) javascript begonnen
| * e103d4b Nav Start                 <<<<<< alter Hash
|/
* b2228f5 Einleitung
* 57085c3 initial
```

8.2 Weniger problemlos

Wenn vom Cherry-Pick eine Datei (oder mehrere) betroffen sind, die in beiden Branches geändert wurden, dann wird die Situation schnell unübersichtlich. Es ensteht wieder eine Art von Merge-Konflikt, den wir auf ganz spezielle Weise auflösen müssen.

Für den Überblick: Das nachfolgende Script nimmt Änderungen an der gleichen Datei in zwei Branches vor. Hier der ist der Inhalt von Script `material/09_cherry/cherry_pick_2.sh`:

```
#!/bin/bash

# In beiden Branches wird die gleiche Datei
# bei jedem Commit um eine Zeile ergänzt.
# "dev" hat 5 commits, main nur 3
#
#  dev             main
#  zeile 1         zeile 1
#  zeile 2         zeile 2
#  zeile 3         zeile 5
#  zeile 4
#  zeile 5
#
# Pickt man Zeile 3 (commit 3) von dev, so liegt
# die Zeile in der Datei nicht in der richtigen Zeile!

set -e
set -x

# putzen
cd
rm -rf cherry_labor

# Repo erstellen
mkdir cherry_labor
cd cherry_labor
git init
git branch -m main

############
# auf main
echo "Zeile 1" > index.html
git add index.html; git commit -m "zeile 1"

echo "Zeile 2" >> index.html
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
git add index.html; git commit -m "zeile 2"

############
# wechsel zu development
git switch -c development  # branch wechsel

echo "Zeile 3" >> index.html
git add index.html; git commit -m "zeile 3"

echo "Zeile 4" >> index.html
git add index.html; git commit -m "zeile 4"

###########
## Zurück zu main
git switch main  # wieder auf main

echo "Zeile 5" >> index.html
git add index.html; git commit -m "zeile 5"


# Darstellen
git log --oneline --decorate --graph --all
```

```
# Ausgabe
* 3aef088 (development) zeile 4
* b287548 zeile 3      <<<<<<<<<< der hier soll
| * 69336b4 (HEAD -> main) zeile 5   <<< da her
|/
* 896aefc zeile 2
* 1fb21e9 zeile 1
```

Mit Hilfe von

```
git cherry-pick b287548
```

Bekommen wir folgenden Konflikt:

```
automatischer Merge von index.html
KONFLIKT (Inhalt): Merge-Konflikt in index.html
error: Konnte b287548... (zeile 3) nicht anwenden
Hinweis: Lösen Sie alle Konflikte manuell auf, markieren Sie diese
Hinweis: mit "git add/rm <Pfadspezifikation>" und führen Sie dann
Hinweis: "git cherry-pick --continue" aus.
Hinweis: Sie können stattdessen auch diesen Commit auslassen, indem
Hinweis: Sie "git cherry-pick --skip" ausführen.
Hinweis: Um abzubrechen und zurück zum Zustand vor
Hinweis: "git cherry-pick" zu gelangen, führen Sie
Hinweis: "git cherry-pick --abort" aus.
```

Nun muss die betroffen Datei per Hand korrigiert und neu gespeichert werden. Das Ziel soll für uns so aussehen:

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
Zeile 1
Zeile 2
Zeile 3
Zeile 5
```

Eine Überprüfung zeigt uns, dass die Datei als *von beiden geändert* geführt wird und wir sie durch `git add` wieder stagen sollen: (Umbrüche angepasst!)

```
Auf Branch main
Sie führen gerade "cherry-pick" von Commit b287548 aus.
  (beheben Sie die Konflikte und führen Sie dann
  "git cherry-pick --continue" aus)
  (benutzen Sie "git cherry-pick --skip", um diesen Patch
  auszulassen)
  (benutzen Sie "git cherry-pick --abort", um die
  Cherry-Pick-Operation abzubrechen)

Nicht zusammengeführte Pfade:
  (benutzen Sie "git add/rm <Datei>...", um die Auflösung
  zu markieren)
    von beiden geändert:    index.html

keine Änderungen zum Commit vorgemerkt
(benutzen Sie "git add" und/oder "git commit -a")
```

Das machen wir

```
git add index.html
git status

# Ausgabe
Auf Branch main
Sie führen gerade "cherry-pick" von Commit b287548 aus.
  (alle Konflikte behoben: führen Sie "git cherry-pick --continue"
  aus)
  (benutzen Sie "git cherry-pick --skip", um diesen Patch
  auszulassen)
  (benutzen Sie "git cherry-pick --abort", um die
  Cherry-Pick-Operation abzubrechen)

Zum Commit vorgemerkte Änderungen:
    geändert:       index.html
```

Es folgt nun **KEIN** normaler Commit, sondern ein `git cherry-pick --continue`, das sich aber wie ein Commit verhält, d.h. wir müssen eine Message eingeben und abspeichern. Nun erst ist der Vorgang abgeschlossen.

Hinweis
Beachte, dass in obigem Beispiel beide Branches komplett committet waren. Es gab also keine aktuell abgeänderten Dateien, was den Ablauf noch komplexer machen würde!

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

8.3 Noch unangenehmer ...

Nun wollen wir 3 Dateien betrachten (datei_1.html,datei_2.html,datei_3.html). Davon soll nur `datei_1.html` im `main`-Branch liegen. Die anderen beiden Dateien werden im `development`-Branch abwechselnd bearbeitet. Wie wollen einen *späteren* Commit cherry-picken. Im Tree sieht das so aus:

```
* 177a8a6 (development) z3 d1
* d46d7bb z2 d2     <<<<<<< wollen wir von hier ...
* 1b07ddb z1 d3
* 8fa5974 z1 d2
| * a4cc92c (HEAD -> main) d1 z3 <<< ... nach da picken
|/
* cc253e1 z2 d1
* 09a1330 z1 d1
```

Erzeugen können wir die Situation mit dem Script `material/09_cherry/cherry_pick_3.sh`:

```
#!/bin/bash

# Erstellen einer Struktur zum Probieren
# von Cherry-pick. Führt i.a. zu einem
# Konflikt im Sinne dass eine Datei
# manuell geadded werden muss

# z2 d3 steht für zeile 2 Datei 3

set -e
set -x

cd
rm -rf cherry_labor

## Lokales Repo erstellen
mkdir cherry_labor
cd cherry_labor
git init
git branch -m main

# Arbeit auf main
echo "Zeile 1" > datei_1.html
git add datei_1.html; git commit -m "z1 d1"

echo "Zeile 2" >> datei_1.html
git add datei_1.html; git commit -m "z2 d1"

############
# Wechsel zu development
git switch -c development  # branch wechsel

echo "Zeile 1" >> datei_2.html
git add datei_2.html; git commit -m "z1 d2"
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
echo "Zeile 1" >> datei_3.html
git add datei_3.html; git commit -m "z1 d3"

echo "Zeile 2" >> datei_2.html
git add datei_2.html; git commit -m "z2 d2"

echo "Zeile 3" >> datei_2.html
git add datei_2.html; git commit -m "z3 d2"

###########
# zurück auf main
git switch main  # wieder auf main

echo "Zeile 3" >> datei_1.html
git add datei_1.html; git commit -m "z3 d1"

# Darstellen
git log --oneline --decorate --graph --all

# Ab hier kann im Terminal experimentiert werden

exit 0
```

Hinweis
Wichtig ist hier nun, dass `cherry-pick` eigentlich *nur* bedeutet:
„Führe den genannten Commit an der aktuellen Stelle aus."
Das ist auch die Erklärung für die nachfolgende Meldung!

Der Befehl `git cherry-pick d46d7bb` liefert diese, etwas seltsame Meldung:

```
KONFLIKT (ändern/löschen): datei_2.html gelöscht in HEAD und
geändert in d46d7bb (z2d2). Stand d46d7bb (z2d2) von datei_2.html
wurde im Arbeitsbereich gelassen.
error: Konnte d46d7bb... (z2d2) nicht anwenden
Hinweis: Lösen Sie alle Konflikte manuell auf, markieren Sie diese
Hinweis: mit "git add/rm <Pfadspezifikation>" und führen Sie dann
Hinweis: "git cherry-pick --continue" aus.
Hinweis: Sie können stattdessen auch diesen Commit auslassen, indem
Hinweis: Sie "git cherry-pick --skip" ausführen.
Hinweis: Um abzubrechen und zurück zum Zustand vor
Hinweis: "git cherry-pick" zu gelangen, führen Sie
Hinweis:  "git cherry-pick --abort" aus.
```

Im betroffenen Commit (nur relevante Zeilen) steht:

```
git show b67e3e2

# Ausgabe
commit d46d7bb...
Author: wolfgang <lehrer@t-online.de>
Date:   Sun Mar 29 14:13:55 2026 +0200
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
    z2 d2

diff --git a/datei_2.html b/datei_2.html
....
--- a/datei_2.html
+++ b/datei_2.html
@@ -1 +1,2 @@
 Zeile 1
+Zeile 2
```

Der Datei `datei_2.html` soll eine weitere Zeile hinzugefügt werden. Aktuell sind wir aber auf dem Branch `main` und dort gibt es die Datei `datei_2.html` aber gar nicht – also kann git auch keine Zeile hinzufügen. Als Notlösung wird die Datei `datei_2.html` mit dem Zustand vom betroffenen Commit in *Working-Directory* von `main` kopiert.
DAS ist die Aussage der der ersten Zeilen – und auch, dass *git* dieses Problem nicht lösen kann.

Durch `git add datei_2.html` bekommen wir dann

```
Auf Branch main
Sie führen gerade "cherry-pick" von Commit d46d7bb aus.
  (alle Konflikte behoben: führen Sie "git cherry-pick --continue"
  aus)
  (benutzen Sie "git cherry-pick --skip", um diesen Patch
  auszulassen)
  (benutzen Sie "git cherry-pick --abort", um die Cherry-Pick-
  Operation abzubrechen)

Zum Commit vorgemerkte Änderungen:
    neue Datei:     datei_2.html
```

und `git cherry-pick --continue` beendet den Prozess nach Eingabe der üblichen Commit-Message.

```
[main a908bef] Picken von z2d2
 Date: Thu Jan 8 07:21:31 2026 +0100
 1 file changed, 2 insertions(+)
 create mode 100644 datei_2.html
```

mit

```
* a908bef (HEAD -> main) Picken von z2d2  <<<< Erfolg
* a4cc92c d1z3
| * 177a8a6 (development) z3d2
| * d46d7bb z2d2
| * 1b07ddb z1d3
| * 8fa5974 z1d2
|/
* cc253e1 z2d1
* 09a1330 z1d1
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Wichtig
Auch wenn im *Working-Directory* zum Zeitpunkt von Commit d46d7bb die Datei `index_3.html` vorhanden ist, so wird sie nicht in den Branch `main` übernommen! Das liegt einfach daran, dass sie im Commit – also im `diff` gar nicht erwähnt wird.

.

Du siehst also, dass das Picken durchaus seine komplexen Seiten hat, weshalb ein Einsatz im Unterricht wohl auch nur sehr bedingt sinnvoll ist!

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

9 Arbeiten im Team

9.1 Herausforderungen

Das Arbeiten im Team stellt dich vor zwei Probleme:

- Welche Server-Infrastruktur sollst du verwenden?
- Was ändert sich am Workflow in *git*

Beide Punkte sollen nachfolgend besprochen werden, wobei es zwangsläufig Überschneidungen geben wird.

9.1.1 Infrastruktur

Im Kern ist es egal ob wir vom Setup im Klassenzimmer oder im Internet sprechen – auch wenn ein Netzwerk beteiligt ist, so bleibt die Basistechnik der letzten Kapitel gleich. Neu sind allerdings Aspekte der Sicherheit (Zugriff) und der Logistik beim Datenabgleich zwischen mehreren Repositories.

Für eine server-basierende Teamarbeit gibt es folgende Varianten (selbst gehostet oder kommerziell):

- einfacher *git*-Server
  Die Installation ist einfach, die Grundkonfiguration auch. Der Zugriff erfolgt über ssh. Soll das ohne Kennwort funktionieren, so müssen Schlüssel erzeugt und auf dem Server eingepflegt werden. Später führt um die Schlüssel aber ohnehin kein Weg herum.
- *git*-Server mit kennwortlosem Zugriff über http.
  Funktioniert, ist aber keinesfalls für den Live-Einsatz über einen längeren Zeitraum im Internet zu empfehlen.
- Ein *git*-Server mit *Gitea*
  Gui-basiertes System mit vielen Optionen und weniger komplex als *GitLab*
- Ein *git*-Server mit *GitLab*
- Ein bezahlter DSGVO-Konformer Anbieter
- Gemeinnützige Organisationen wie *codeberg.org* (kein CICD(?))

9.1.2 Änderungen am Workflow

Bei der Arbeit im Team ändert sich im Vergleich zum lokalen Arbeiten zunächst wenig. Die Befehle *branch, add, commit, …* funktionieren genau wie bisher. Neu sind allerdings die folgenden Schritte

- Initiales Holen des Repos auf den eigenen Rechner (=clone)
- Regelmäßiges Abrufen des aktuellen Standes (=pull)
- Veröffentlichen des eigenen Standes (=push)

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Hierbei sind einige Spielregeln zu beachten, damit es nicht zu Problemen kommt. Diese Regeln werden meist als Workflow oder im Speziellen auch als GitFlow bezeichnet. Es gibt hier zwar gewisse – aber keinesfalls verbindliche – Standards. Jede Firma stellt für *git* ihre eigenen, fest vorgeschriebenenen Abläufe auf, an die sich die Mitarbeiter zu halten haben.

9.1.3 Mögliche Vereinbarungen

Auch die folgenden Vereinbarungen sind nur exemplarisch zu sehen und können in jeder Firma oder jedem Projekt anders umgesetzt werden!

Verantwortung
Im Projekt gibt es einen Branch `main`, der aber durch entsprechende Maßnahmen für *einfache* Mitarbeiter schreibgeschützt ist. Änderungen dürfen nur nach gründlichen Tests und Code-Reviews in diesen Zweig aufgenommen werden. In der Regel gilt hier auch ein *Mehr-Augen-Prinzip* oder noch weiter gestaffelte Zuständigkeitshierarchien.

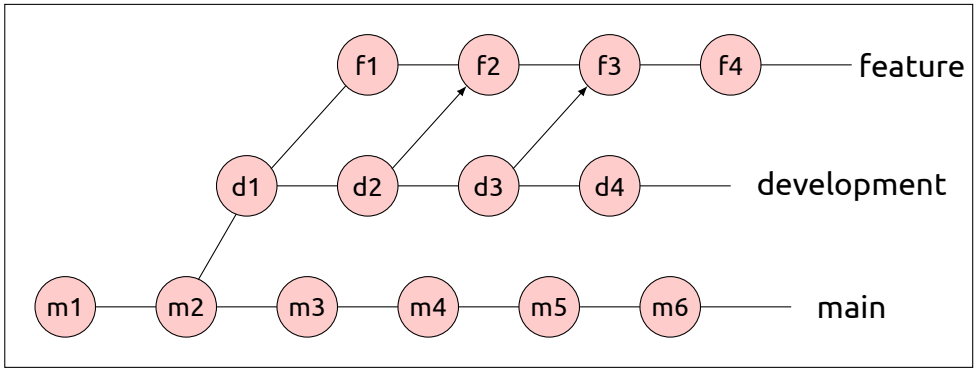
Arbeitsbranch
Je nach Größe des Projekt-Teams gibt es einen oder mehrere `development`-Branches. Die Mitarbeiter holen sich diesen Zweig und lassen von ihm ihre eigenen Arbeitszweige (Feature-Branches, Bugfix-Branches) ausgehen. Ist ihre Arbeit dort beendet, muss der Code getestet werden und dann erst erfolgt der Merge in den `development`-Branch.

9.1.3.1 Etwas mehr Details

Da im Team mehrere Personen zu unvorhersehbaren Zeitpunkten Änderungen am Code vornehmen, muss jeder Mitarbeiter dafür sorgen, dass er immer die aktuellste Version vorliegen hat. Da er seine Entwicklung aber in seinem eigenen `Feature`-Branch vorantreibt, müssen diese Änderungen der Kollegen dort aber erst ankommen – dies geschieht durch regelmäßige Merges der fremden Arbeit **in** den eigenen Branch.

Beispiel

Aus Platzgründen zeichne ich das Branch-Diagramm hier waagrecht. Man sieht, dass beim Commit m_2 der `devlopment`-Branch abzweigt und dann vom Commit d_1 sofort der `feature`-Branch.
Ab jetzt werden regelmäßig Änderungen am `development`-Branch auf den eigenen `feature`-Branch gezogen, um so immer den aktuellen Stand zu haben. Diese Änderungen am `development`-Branch können von den Teamkollegen stammen.



Merges in die andere Richtung – also vom `feature`- auf den `development`-Branch sieht man im Bild oben noch keine. Auch der `main`-Branch wurde noch nie aktualisiert.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

9.1.3.2 Pull und fetch

Die Grundidee von *holen* und *veröffentlichen* ist relativ einfach und eventuell muss man den Schülern auch nicht mehr erzählen. Da aber Code von anderen Entwicklern auf deinen Rechner kommt, solltest du diesen eventuell nicht ungesehen in deine Entwicklung aufnehmen. Der Befehl `git pull` macht aber genau das.
Willst du an dieser Stelle sicher gehen, so führst du zuerst `git fetch` aus und siehst dir die Änderungen an – siehe weiter unten. Im Anschluss kannst du sie dann mit `git merge` übernehmen.

Wo sind die Dateien?
Was am Anfang sehr geheimnisvoll ist, betrifft den Speicherort der Dateien, die mit `git fetch` geholt wurden. Natürlich müssen sie in irgendeiner Weise im Ordner `.git` abgelegt sein, aber wie sieht das aus?

Dafür brauchen wir nun einen etwas *klobigeren* Setup, bestehend aus

- einem Server-Repository
- einem lokalen Repository (Kollege 1)
- einem lokalen Repository (Kollege 1)

Wir können das aber problemlos auf dem eigenen Rechner simulieren!

```
set -x
cd /tmp
rm -rf fetch_lab
mkdir fetch_lab
cd fetch_lab

git init --bare server.git
cd server.git
git branch -m main

cd ..
git clone ./server.git kollege_1
git clone ./server.git kollege_2
```

Unser Ordner sieht nun so aus:

```
ls

# Ausgabe
kollege_1  kollege_2  server.git
```

Nun erstellt Kollege 1 eine Datei und pusht sie ins Server-Repository. Beachte, dass der Autor hier nicht aussagekräftig ist, weil wir das ja als globale Einstellung für *git* festgelegt haben!

```
cd kollege_1
echo "Zeile 1" > datei.txt
git add . && git commit -m "Inhalt von Kollege 1"
git push
cd ..
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
# Ausgabe
[main (Root-Commit) a082f7f] Inhalt von Kollege 1
 1 file changed, 1 insertion(+)
 create mode 100644 datei.txt
Objekte aufzählen: 3, fertig.
Zähle Objekte: 100% (3/3), fertig.
Schreibe Objekte: 100% (3/3), 227 Bytes | 227.00 KiB/s, fertig.
Gesamt 3 (Delta 0), Wiederverwendet 0 (Delta 0), Pack wiederverwendet 0
To /tmp/fetch_lab/./server.git
 * [new branch]      main -> main
```

Kollege 2 kann diesen Stand nun abholen:

```
cd kollege_2
git pull

# Ausgabe
remote: Objekte aufzählen: 3, fertig.
remote: Zähle Objekte: 100% (3/3), fertig.
remote: Gesamt 3 (Delta 0), Wiederverwendet 0 (Delta 0), Pack wiederverwendet 0
Entpacke Objekte: 100% (3/3), 207 Bytes | 207.00 KiB/s, fertig.
Von /tmp/fetch_lab/./server
 * [neuer Branch]    main       -> origin/main
```

Die Datei ist nun angekommen und kann verändert werden:

```
echo "Zeile 2 von Kollege 2" >> datei.txt
git add .
git commit -m "Änderung"

# Ausgabe
[main 9654bc5] Änderung
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Nun kommen die Änderungen auf den Server

```
git push
cd ..

# Ausgabe
Objekte aufzählen: 5, fertig.
Zähle Objekte: 100% (5/5), fertig.
Schreibe Objekte: 100% (3/3), 265 Bytes | 265.00 KiB/s, fertig.
Gesamt 3 (Delta 0), Wiederverwendet 0 (Delta 0), Pack wiederverwendet 0
To /tmp/fetch_lab/./server.git
   a082f7f..9654bc5  main -> main
```

Bevor Kollege 1 nun einen *fetch* ausführt, werfen wir einen Blick in seinen *git*-Ordner:

```
cd kollege_1
tree .git
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
├── index
├── info
│   └── exclude
├── logs
│   ├── HEAD
│   └── refs
│       ├── heads
│       │   └── main
│       └── remotes
│           └── origin
│               └── main
├── objects
│   ├── 4d
│   │   └── 5819dc3711611c3e6a16b1ed362a6952e980fc
│   ├── a0
│   │   └── 82f7f5b85c95d6a90f048c3a104c7901b944f7
│   ├── b1
│   │   └── 367440191d3abc86bb46f955d140fec7eef42a
│   ├── info
│   └── pack
└── refs
    ├── heads
    │   └── main
    ├── remotes
    │   └── origin
    │       └── main
```

Nun führt er einen *fetch* aus:

```
git fetch

# Ausgabe
remote: Objekte aufzählen: 5, fertig.
remote: Zähle Objekte: 100% (5/5), fertig.
remote: Gesamt 3 (Delta 0), Wiederverwendet 0 (Delta 0), Pack wiederverwendet 0
Entpacke Objekte: 100% (3/3), 245 Bytes | 245.00 KiB/s, fertig.
Von /tmp/fetch_lab/./server
   a082f7f..9654bc5  main       -> origin/main
```

```
├── logs
│   ├── HEAD
│   └── refs
│       ├── heads
│       │   └── main
│       └── remotes
│           └── origin
│               └── main
├── objects
│   ├── 4d
│   │   └── 5819dc3711611c3e6a16b1ed362a6952e980fc
│   ├── 62
│   │   └── e8ea14d475f7f333fda2f6a2f2e75f81e177c1
│   ├── 96
│   │   └── 54bc5747f1917cf67d6a4ec4e33fc5103087b5
│   ├── a0
│   │   └── 82f7f5b85c95d6a90f048c3a104c7901b944f7
│   ├── b1
│   │   └── 367440191d3abc86bb46f955d140fec7eef42a
│   ├── e0
│   │   └── 6235b010b363d46fd6f42512eca28558c2cd73
│   ├── info
│   └── pack
└── refs
    ├── heads
    │   └── main
    ├── remotes
    │   └── origin
    │       └── main
```

Es sind also drei *git*-Objekte dazu gekommen. Das sind die Änderungen von Kollege 2, die noch nicht in die Arbeit von Kollege 1 übernommen wurden.

Das kann man im Log auch schön sehen:

```
git log --all --oneline

# Ausgabe
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
9654bc5 (origin/main) Änderung
a082f7f (HEAD -> main) Inhalt von Kollege 1
```

Um die Änderungen auch zu sehen, gibt es wieder *diff*:

```
git diff a082f7f  9654bc5

# Ausgabe
diff --git a/datei.txt b/datei.txt
index b136744..62e8ea1 100644
--- a/datei.txt
+++ b/datei.txt
@@ -1 +1 @@
-Zeile 1
+Zeile 2 von Kollege 2
```

Findet die Änderung Gefallen, so kann Kollege 1 sie mit `git merge` in seine Arbeit übernehmen. Falls er sich nicht sicher ist, wäre ein Branch an dieser Stelle eine Alternative. Der Befehl dafür hätte gelautet: `git switch -c abchecken origin/main`. Hier kann er nun die Änderungen auch testen und falls alles passt in `main` wieder `git merge` aufrufen.

9.1.3.3 Was kann schief gehen?

Angenommen, beide Kollegen arbeiten fleißig und ändern Dinge in ihrem Repository. Dann können wir 2 Fälle unterscheiden:

- Arbeit an verschiedenen Dateien
- Arbeit an der gleichen Datei

Wir wollen das in dieser Reihenfolge durchspielen.

```
cd kollege_1
echo "Datei von Kollege 1" > koll_1_datei.txt
git add .
git commit -m "Neue Datei koll 1"
git push
cd ..

# Ausgabe
[main 7b493ff] Neue Datei koll 1
 1 file changed, 1 insertion(+)
 create mode 100644 koll_1_datei.txt
Objekte aufzählen: 4, fertig.
Zähle Objekte: 100% (4/4), fertig.
Delta-Kompression verwendet bis zu 6 Threads.
Komprimiere Objekte: 100% (2/2), fertig.
Schreibe Objekte: 100% (3/3), 304 Bytes | 304.00 KiB/s, fertig.
Gesamt 3 (Delta 0), Wiederverwendet 0 (Delta 0), Pack wiederverwendet 0
To /tmp/fetch_lab/./server.git
   9d502b1..7b493ff  main -> main
```

Kollege 2 macht die gleiche Aktion auf einer anderen Datei

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
cd kollege_2
echo "Datei von Kollege 2" > koll_2_datei.txt
git add .
git commit -m "Neue Datei koll 2"
git push

# Ausgabe

[main b3ca883] Neue Datei koll 2
 1 file changed, 1 insertion(+)
 create mode 100644 koll_2_datei.txt
To /tmp/fetch_lab/./server.git
 ! [rejected]        main -> main (fetch first)
error: Fehler beim Versenden einiger Referenzen nach '/tmp/fetch_lab/./server.git'
Hinweis: Updates were rejected because the remote contains work that you do not
Hinweis: have locally. This is usually caused by another repository pushing to
Hinweis: the same ref. If you want to integrate the remote changes, use
Hinweis: 'git pull' before pushing again.
Hinweis: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Git weigert sich also, weil die Änderungen von Kollege 1 noch nicht *abgeholt* wurden. Es ist demnach sinnvoll, vor jedem *push* (und auch bei Arbeitsbeginn) zunächst einen *pull* auszuführen. Es kann aber noch komplexer werden:

```
git pull

# Ausgabe
remote: Objekte aufzählen: 4, fertig.
remote: Zähle Objekte: 100% (4/4), fertig.
remote: Komprimiere Objekte: 100% (2/2), fertig.
remote: Gesamt 3 (Delta 0), Wiederverwendet 0 (Delta 0), Pack wiederverwendet 0
Entpacke Objekte: 100% (3/3), 284 Bytes | 142.00 KiB/s, fertig.
Von /tmp/fetch_lab/./server
   9d502b1..7b493ff  main       -> origin/main
Hinweis: Sie haben abweichende Branches und müssen angeben, wie mit diesen
Hinweis: umgegangen werden soll.
Hinweis: Sie können dies tun, indem Sie einen der folgenden Befehle vor dem
Hinweis: nächsten Pull ausführen:
Hinweis:
Hinweis:   git config pull.rebase false  # Merge
Hinweis:   git config pull.rebase true   # Rebase
Hinweis:   git config pull.ff only       # ausschließlich Vorspulen
Hinweis:
Hinweis: Sie können statt "git config" auch "git config --global" nutzen, um
Hinweis: einen Standard für alle Repositories festzulegen. Sie können auch die
Hinweis: Option --rebase, --no-rebase oder --ff-only auf der Kommandozeile nutzen,
Hinweis: um das konfigurierte Standardverhalten pro Aufruf zu überschreiben.
fatal: Es muss angegeben werden, wie mit abweichenden Branches umgegangen werden sollen.
```

Das sieht nun sehr bedrohlich aus … was ist passiert?

Betrachten wir die Situation der Kollegen im Detail, **bevor** das passiert ist. Dafür solltest du das Script `material/git_core/team_hands_on/kollegen.sh` laufen lassen.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

```
set
cd /tmp
rm -rf fetch_lab
mkdir fetch_lab
cd fetch_lab

# Server-Repo erstellen
git init --bare server.git
cd server.git
git branch -m main

# Die Kollegen clonen
cd ..
git clone ./server.git kollege_1
git clone ./server.git kollege_2

# Kollege 1 arbeitet
cd kollege_1
echo "Zeile 1" > datei.txt
git add . && git commit -m "Inhalt von Kollege 1"
git push
cd ..

# Kollege 2 holt ab
cd kollege_2
git pull

# Kollege 2 arbeitet
echo "Zeile 2 von Kollege 2" >> datei.txt
git add .
git commit -m "Änderung"
git push
cd ..

# Kollege 1 holt ab
cd kollege_1
git pull

# Kollege 1 arbeitet weiter
echo "Datei von Kollege 1" > koll_1_datei.txt
git add .
git commit -m "Neue Datei koll 1"
cd ..

# Kollege 2 arbeitet weiter
cd kollege_2
echo "Datei von Kollege 2" > koll_2_datei.txt
git add .
git commit -m "Neue Datei koll 2"

echo "Unterschiedliche Stände in verschiedenen Dateien"
```

Aktueller Stand nach dem Script:

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Kollege 1

```
cd kollege_1
git log --oneline --graph --all --decorate

# Ausgabe
* 7658af2 (HEAD -> main) Neue Datei koll 1
* b34f432 (origin/main) Änderung
* 3a1b032 Inhalt von Kollege 1
```

Kollege 2

```
cd ../kollege_2
git log --oneline --graph --all --decorate

# Ausgabe
* 493aad8 (HEAD -> main) Neue Datei koll 2
* b34f432 (origin/main) Änderung
* 3a1b032 Inhalt von Kollege 1
```

Beide haben den Commit mit dem Hash `b34f432`, der den Stand auf dem Server darstellt. Danach haben sie beide weiter gemacht und an diesen Commit einen neuen Commit gehängt. Diese Commits haben nun unterschiedliche Hashes. *Git* steht also vor dem Problem, was es in dieser Situation machen soll. Es ist nicht möglich, die History mit 2 *Köpfen* fortzuführen. Die beiden Commits müssen zusammengeführt werden und dafür gibt es eben 3 Möglichkeiten

- Nie Rebase (= Merge)
- Immer Rebase (= Rebase)
- Nur Fast Forward

Dem typischen *pull* von *git* entspricht am ehesten die erste Variante – deshalb sollte sie auch eingestellt werden (außer man weiß genau, was man macht)

Nach dem üblichen Editor-Fenster zum erstellen einer Commit-Message werden die beiden Zweige von Kollege 1 und Kollege 2 zusammengeführt und Kollege 2 kann einen *push* ausführen.

9.1.4 Einige Fragen zu push und pull

Wie kann ich einen neuen Branch pushen?

```
# auf dem Client
git switch -c arbeit
git push -u origin arbeit
```

Hinweis
Du hast nun mehrfach *origin main* aber auch *origin/main* gesehen. Was ist der Unterschied? Die erste Variante sind Argumente für die *git*-Befehle. *Origin* ist die Adresse des Servers, *main* ist der betroffene Branch.
Bei der zweiten Variante handelt es sich um eine Referenz auf dem Client, die den Branch auf dem Server beschreibt.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

Ist es egal, in welchem Branch ich fetch/pull/push ausführe?
Jein.
Beim Pushen kann nicht viel schief gehen. Upstream-Branches laufen automatisch und wenn sie nicht als upstream konfiguriert sind, dann geht der Push (standardmäßig) nicht.

Ein `git pull` holt sich auch immer den Inhalt des aktuellen Branches (sofern er auf dem Server existiert).

Gefährlicher ist es, wenn du zum Beispiel im Branch `arbeit` bist ein `git pull versuche` ausführst. In diesem Fall wird der falsche Branch geholt in deinen hinein gemerged.

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.

10 Anhang

10.1 Installation von Tree

- Geh auf die Seite http://gnuwin32.sourceforge.net/packages/tree.htm und lade dir das *zip* des Binaries herunter.
- Entpacke die zip-Datei und kopiere die Datei *tree* im Ordner *bin* in die Zwischenablage
- Lege im Home deines Benutzers einen Ordner *bin* an und kopiere die Datei *tree* hinein.

Hinweis Der bin-Ordner muss an folgender Stelle erzeugt werden:

```
c:\users\benutzername\bin
```

© 2026 by Wolfgang Höfer, licensed under CC BY-SA 4.0.